



Politechnika Warszawska

**Wydział Matematyki
i Nauk Informacyjnych**

Reprezentacja Wiedzy

Procesy dziaa zoonych

Punkt kontrolny 2

Zespół projektowy nr 4:

Wojciech Jasiski
Krzysztof Grybo
Jarosław Bieniek
Filip Filipkowski
Sebastian Rydz
Jakub Pietrzak
Sandra Adamiec

Warszawa, 23 kwietnia 2026

Spis treści

1	Wprowadzenie	3
1.1	Cel projektu	3
1.2	Klasa $\mathbb{DS4}$ – warunki Z1–Z7	3
1.3	Mapa zapoycze z wykadów	3
1.4	Ograniczenia $\mathbb{DS4}$	3
2	Podstawy formalne	4
2.1	Sygnatura	4
2.2	Formuy zdaniowe	4
2.3	Stany	5
2.4	Literay i DNF	5
3	Jzyk akcji — skadnia	5
3.1	Zdania dziedziny — tabela zbiorcza	6
3.2	Pena BNF dziedziny	6
3.3	Opis poszczgólnych zda	7
3.4	Dziedzina D	8
4	Semantyka — poziom akcji prostej	8
4.1	Stany dopuszczalne Σ	8
4.2	Stany pocztkowe Σ_0	9
4.3	Funkcja Res_0	9
4.4	Funkcja New	10
4.5	Minimalizacja — $\text{Res}(A, \sigma)$	10
4.6	Niewykonalno	10
5	Semantyka — akcje zoone	11
5.1	Akcja zoona $\mathbb{A}$	11
5.2	Konflikty	11
5.3	Dekompozycje $\Delta(\mathbb{A}, \sigma)$	12
5.4	$\text{Res}(\mathbb{A}, \sigma)$ przez dekompozycje	13
5.5	Uwaga o „restricted inheritance”	14
5.6	Porównanie z treci projektu	14
6	Procesy i model dziedziny	15
6.1	Proces $\mathbb{P}$	15
6.2	Funkcja Ψ	15
6.3	Prawdziwo zda wartociowych	16
6.4	Model dziedziny	16
6.5	Jednoznaczno modelu	17
7	Jzyk kwerend	17
7.1	Zdania jzyka kwerend	17
7.2	Skadnia	18
7.3	Semantyka formalna	18
8	Decyzje projektowe i uzasadnienia	19
8.1	Σ_0 zamiast σ_0	19
8.2	Mechanizm AC zamiast rozcznoci z treci projektu	19
8.3	$\text{Res}(\mathbb{A}, \sigma) = \emptyset$ przy braku wykonalnych skadowych	20
8.4	Semantyka przypadków brzegowych	20

9	Algorytmizacja (pseudokod)	20
9.1	Generacja Σ	20
9.2	Res dla akcji prostej	21
9.3	Generacja Σ_0	23
9.4	Wykrywanie konfliktów	24
9.5	Generacja dekompozycji	25
9.6	Res dla akcji zoonej	26
9.7	Ewaluacja kwerend	27
9.8	Uwagi o zoonoci	27
10	Przykady	28
10.1	YSP z rosyjsk ruletk	28
10.2	Producent i konsumenci	29
10.3	Dwa przeczniki, arówka i alarm	31
10.4	Dwóch robotników wynoszących stó	33
11	Podsumowanie i kierunek dalszych prac	37

1 Wprowadzenie

1.1 Cel projektu

Zadanie polega na zaprojektowaniu dwóch formalnych języków dla klasy $\mathbb{DS4}$ systemów dynamicznych:

- *języka akcji* — służy do opisu dziedziny (fluenty, akcje, ich skutki, ograniczenia),
- *języka kwerend* — pozwalającego zadawać pytania o wykonalność procesów i osiągnięcie celów.

Klasa $\mathbb{DS4}$ czy cechy kilku języków akcji znanych z wykładów. Mechanizmy inercji, niedeterminizmu, ramifikacji i niewykonalności pochodzą z języka AR (wykład 2). Obsługa akcji zoonych projektujemy w oparciu o język AC (wykład 5) — adaptujemy mechanizm detekcji konfliktów i dekompozycji do warunków $\mathbb{DS4}$, w których opisy dziedzin zawierają wyłącznie akcje proste.

1.2 Klasa $\mathbb{DS4}$ – warunki Z1–Z7

Systemy klasy $\mathbb{DS4}$ są zdefiniowane przez siedem fundamentalnych założeń, które określają ich semantykę i możliwości modelowania:

1. **Z1. Prawo inercji** fluenty, które nie są bezpośrednio lub pośrednio zmieniane przez akcję, zachowują swoją wartość w kolejnym stanie (np. jeśli akcja "malowanie ciany" zmienia tylko kolor, to położenie drzwi pozostaje bez zmian).
2. **Z2. Pełna informacja o strukturze** system posiada dostęp do kompletnej wiedzy o zbiorze fluentów F , akcji A oraz wszystkich ich potencjalnych skutkach zdefiniowanych w modelu.
3. **Z3. Niedeterminizm działań** wykonanie pojedynczej akcji w konkretnym stanie może prowadzić do wielu alternatywnych stanów wynikowych (np. rzut monet skutkuje orłem lub reszką).
4. **Z4. Warunki wstępne i wynikowe** każda akcja posiada zdefiniowaną formułę logiczną, która musi być spełniona przed jej wykonaniem; w przypadku niespełnienia warunku wstępnego, akcja nie wywołuje żadnych zmian (efekt pusty).
5. **Z5. Niewykonalność akcji** model dopuszcza istnienie stanów, w których dana akcja jest całkowicie zablokowana i nie może zostać podjęta przez agenta.
6. **Z6. Warunki integralności (Ramifikacje)** globalne reguły logiczne nakładane na stany wpływają na skutki uboczne akcji; zmiana jednego fluentu może wymusić zmiany innego, aby zachować spójność systemu.
7. **Z7. Opis częściowy** język pozwala na specyfikację stanów, w których wartości niektórych fluentów są nieznane, co dotyczy zarówno stanu początkowego, jak i stanów osiągniętych po sekwencjach akcji.

1.3 Mapa zapożyczeń z wykładów

1.4 Ograniczenia $\mathbb{DS4}$

Pomimo wysokiego stopnia zaawansowania, klasa $\mathbb{DS4}$ wprowadza pewne uproszczenia względem pełnej specyfikacji języka AC oraz rozszerzonych wersji AR , co ma na celu ujednolicenie semantyki przy zachowaniu pełnej mocy wyrazowej dla procesów zoonych. Do kluczowych ograniczeń należą:

Tabela 1: Mapowanie mechanizmów klasy $\mathbb{DS4}$ na materiały teoretyczne.

Mechanizm klasy $\mathbb{DS4}$	ródowy język	Wykład / Materiały
Inercja, ramifikacje, niedeterminizm	AR	Wykład 2
Niewykonalność akcji	AR	Wykład 2
Semantyka kwerend (<i>necessary/possibly</i>)	$AR / \mathcal{L}$	Wykład 3
Konflikty i dekompozycja akcji zoonych	AC	Wykład 5
Opisy częściowe i niepełna informacja	AR	Wykład 2

- **Ograniczenie opisu dziedziny do akcji prostych** — w przeciwieństwie do pełnego języka AC , gdzie akcje zoone mogłyby być definiowane jako stae w opisie dziedziny, $\mathbb{DS4}$ dopuszcza w specyfikacji wyłącznie akcje proste $A \in \mathcal{A}$. Akcje zoone A pojawiają się dopiero na poziomie języka kwerend jako zbiory akcji prostych wykonywanych symultanicznie, co wymusza dynamiczną weryfikację konfliktów zmiennych w trakcie analizy procesu.
- **Brak jawnej klauzuli *impossible* dla zbiorów akcji** — klasa $\mathbb{DS4}$ nie przewiduje bezpośredniego definiowania zakazu współbieżności dla konkretnych par akcji (tzw. *non-concurrent actions*), co jest obecne w niektórych rozszerzeniach języka AC . Wykonalność zbioru akcji jest tu determinowana przez warunek Z5 (lokalna wykonalność składowych) oraz przez mechanizm detekcji konfliktów i dekompozycji zaczerpnięty z języka AC (patrz §5, uzasadnienie w §8).
- **Brak mechanizmu dziedziczenia restrykcyjnego (*restricted inheritance*)** — w $\mathbb{DS4}$ nie stosuje się mechanizmów automatycznego dziedziczenia skutków akcji prostych przez akcje zoone w sposób hierarchiczny. Każda akcja zoona jest dekomponowana „od zera” do poziomu akcji prostych, a ich skutki są agregowane zgodnie z prawem inercji i warunkami integralności.

Powyższe uproszczenia nie ograniczają ogólności systemu w kontekście założeń Z1–Z7, ponieważ każdą niedozwoloną kombinację akcji można zamodelować poprzez odpowiednie zdefiniowanie warunków integralności (Z6) lub warunków wstępnych (Z4), które w przypadku konfliktu logicznego uczynią proces niewykonalnym (Z5). Mechanizm detekcji konfliktów na poziomie zmiennych w sposób naturalny zastępuje jawną definicję *impossible*, zapewniając spójność fizycznej wiata dynamicznego.

2 Podstawy formalne

2.1 Sygnatura

Podstawą formalnego opisu systemów w klasie $\mathbb{DS4}$ stanowi sygnatura Υ , która definiuje słownik pojęć używanych do opisu wiata i zachodzących w nim zmian. Przyjmujemy definicję $\Upsilon = (\mathcal{F}, \mathcal{A})$, gdzie:

- $\mathcal{F}$ jest skończonym zbiorem **fluentów** (zmiennych zdaniowych), które opisują stan wiata;
- $\mathcal{A}$ jest skończonym zbiorem **akcji prostych**.

Wanym rozszerzeniem względem standardowej sygnatury języka AR jest to, że w klasie $\mathbb{DS4}$ zbiór $\mathcal{A}$ zawiera wyłącznie akcje proste — akcje zoone nie stanowią elementów sygnatury pierwotnej i są konstruowane dynamicznie w kwerendach.

2.2 Formuły zdaniowe

Język $\mathcal{L}(\Upsilon)$ umożliwia budowanie zoonych warunków (prekondycji, skutków, wizów) za pomocą formuł zdaniowych. Gramatykę w notacji BNF dla formuły α nad sygnaturą Υ definiujemy następująco:

$$\langle formula \rangle ::= f \mid \neg \langle formula \rangle \mid \langle formula \rangle \wedge \langle formula \rangle \mid \langle formula \rangle \vee \langle formula \rangle \mid \langle formula \rangle \rightarrow \langle formula \rangle$$

gdzie $f \in \mathcal{F}$, a symbole $\top$ oraz $\perp$ oznaczają odpowiednio prawdę (tautologię) i fałsz (sprzeczność).

2.3 Stany

Stanem σ nazywamy funkcję wartościowania (totalną), która każdemu fluentowi ze zbioru $\mathcal{F}$ przypisuje dokładnie jedną wartość logiczną: $\sigma: \mathcal{F} \rightarrow \{0, 1\}$. Relację spełnialności formuły α w stanie σ , oznaczaną jako $\sigma \models \alpha$, definiujemy rekurencyjnie zgodnie z klasycznym logiką zdań:

- $\sigma \models f$ wtw., gdy $\sigma(f) = 1$;
- $\sigma \models \neg\alpha$ wtw., gdy $\sigma \not\models \alpha$;
- $\sigma \models \alpha \wedge \beta$ wtw., gdy $\sigma \models \alpha$ oraz $\sigma \models \beta$;
- $\sigma \models \alpha \vee \beta$ wtw., gdy $\sigma \models \alpha$ lub $\sigma \models \beta$;
- $\sigma \models \alpha \rightarrow \beta$ wtw., gdy $\sigma \not\models \alpha$ lub $\sigma \models \beta$.

2.4 Literały i DNF

W analizie skutków akcji i wykrywaniu konfliktów w klasie $\mathbb{DS4}$ kluczowe znaczenie ma postać normalna formuły:

- **Litera** to fluent f lub jego negacja $\neg f$;
- **dysjunkcyjna postać normalna (DNF)** to alternatywa jednej lub wielu koniunkcji literałów (składników).

Algorytm konwersji formuły α do DNF obejmuje następujące kroki:

1. eliminacja operatorów $\rightarrow$ oraz $\leftrightarrow$ na rzecz $\neg$, $\wedge$, $\vee$;
2. przesunięcie negacji do poziomu fluentów przy użyciu praw De Morgana i podwójnego przeczenia;
3. zastosowanie prawa dystrybucji koniunkcji względem alternatywy.

Przykład. Konwersja formuły $(\neg p \rightarrow q) \wedge r$:

1. eliminacja implikacji: $(\neg(\neg p) \vee q) \wedge r$;
2. podwójna negacja: $(p \vee q) \wedge r$;
3. dystrybucja: $(p \wedge r) \vee (q \wedge r)$.

Wynikowa formuła jest w postaci DNF i składa się z dwóch składników: $C_1 = \{p, r\}$ oraz $C_2 = \{q, r\}$. Takie rozbięcie jest niezbędne do sprawdzania warunku rozłączności zbiorów zmiennych przy akcjach zoonych.

3 Język akcji — skadnia

Projektowany język akcji służy do opisu dziedzin należących do klasy $\mathbb{DS4}$ systemów dynamicznych. Zgodnie z trzecim zadaniem, w opisach dziedzin występują wyłącznie akcje proste, tj. elementy ustalonego zbioru akcji $\mathcal{A}$. Akcje zoone, rozumiane jako zbiory akcji prostych wykonywanych wspólnie, nie należą zatem do skadni języka opisu dziedziny, lecz pojawiają się dopiero na poziomie języka kwerend, w którym budowane są procesy. Oznacza to, że na poziomie opisu dziedziny zachowujemy rdzę języka AR, natomiast mechanizmy związane ze współbiciem są wprowadzane dopiero przy interpretacji procesów i odpowiadających im zapytań.

Niech $\Upsilon = (\mathcal{F}, \mathcal{A})$ będzie sygnaturą, gdzie $\mathcal{F}$ jest skończonym, niepustym zbiorem fluentów, natomiast $\mathcal{A}$ jest skończonym zbiorem akcji prostych. Formuły języka budowane są nad fluentami ze zbioru $\mathcal{F}$ przy użyciu spójników klasycznego rachunku zdań. W szczególności przez α , β oraz π będziemy oznaczać dowolne formuły zdaniowe nad $\mathcal{F}$, za przez f — dowolny fluent należący do $\mathcal{F}$.

3.1 Zdania dziedziny — tabela zbiorcza

W projektowanym języku akcji dopuszczamy osiem podstawowych typów zda. Pierwsze pi z nich opisuje lokaln dynamik lub struktur dziedziny, natomiast trzy ostatnie peni rol zda specyfikacyjnych, odnoszacych si do stanu pocztkowego albo do wasnoci zachodzacych po wykonaniu cigu akcji prostych. Zestawienie wszystkich dopuszczalnych konstrukcji przedstawia tabela 2.

Tabela 2: Typy zda języka akcji dla klasy DS4

Zdanie (zapis formalny)	Znaczenie intuicyjne	ródo
$A \text{ causes } \alpha \text{ if } \pi$	Wykonanie akcji prostej A w stanie speniającym π wymusza zajcie efektu α .	AR
$A \text{ releases } f \text{ if } \pi$	Wykonanie akcji A w stanie speniającym π zwalnia fluent f spod dziaania prawa inercji.	AR
$\text{impossible } A \text{ if } \pi$	Akcja prosta A jest niewykonalna w kadym stanie speniającym π .	AR
$\text{always } \alpha$	Formua α musi by prawdziwa w kadym dopuszczalnym stanie dziedziny.	AR
$\text{noninertial } f$	Fluent f nie podlega prawu inercji.	AR
$\text{initially } \alpha$	Formua α opisuje wiedz o stanie pocztkowym.	AR
$\alpha \text{ after } A_1, \dots, A_n$	Po wykonaniu cigu akcji prostych $A_1, \dots, A_n$ formua α zachodzi zawsze.	AR
$\text{observable } \alpha \text{ after } A_1, \dots, A_n$	Po wykonaniu cigu akcji prostych $A_1, \dots, A_n$ formua α moe zachodzi.	AR

W kadym z powyszych schematów zakadamy, e $A, A_1, \dots, A_n \in \mathcal{A}$, a zatem wszystkie wystpuje akcje s akcjami prostymi. Jest to zgodne z warunkami klasy DS4 i odrónia poziom opisu dziedziny od poziomu języka kwerend, na którym dopiero dopuszczane s akcje zoone.

3.2 Pena BNF dziedziny

Skadni języka akcji definiujemy nastpujco. Przez α, β oraz π oznaczamy dowolne formuy zdaniowe nad zbiorem fluentów $\mathcal{F}$, za przez A dowoln akcj prost nalec do $\mathcal{A}$.

$$\begin{aligned}
\langle \text{dziedzina} \rangle &::= \langle \text{zdanie} \rangle \mid \langle \text{zdanie} \rangle ";" \langle \text{dziedzina} \rangle \\
\langle \text{zdanie} \rangle &::= \langle \text{efekt} \rangle \mid \langle \text{zwolnienie} \rangle \mid \langle \text{niewykonalnosc} \rangle \mid \langle \text{ograniczenie} \rangle \\
&\quad \mid \langle \text{nieinercyjnosc} \rangle \mid \langle \text{poczatkowe} \rangle \mid \langle \text{wartosciowe} \rangle \mid \langle \text{obserwacyjne} \rangle \\
\langle \text{efekt} \rangle &::= A \text{ "causes" } \alpha \text{ "if" } \pi \\
\langle \text{zwolnienie} \rangle &::= A \text{ "releases" } f \text{ "if" } \pi \\
\langle \text{niewykonalnosc} \rangle &::= \text{"impossible" } A \text{ "if" } \pi \\
\langle \text{ograniczenie} \rangle &::= \text{"always" } \alpha \\
\langle \text{nieinercyjnosc} \rangle &::= \text{"noninertial" } f \\
\langle \text{poczatkowe} \rangle &::= \text{"initially" } \alpha \\
\langle \text{wartosciowe} \rangle &::= \alpha \text{ "after" } A_1, \dots, A_n \\
\langle \text{obserwacyjne} \rangle &::= \text{"observable" } \alpha \text{ "after" } A_1, \dots, A_n
\end{aligned}$$

Powysza gramatyka obejmuje wszystkie typy zda dopuszczalnych w języku akcji dla klasy $\mathbb{DS4}$. Warto zauważyć, że składnia nie zawiera żadnych reguł odnoszących się bezpośrednio do akcji zoonych. Ograniczenie to ma charakter zamierzony i wynika z założeń zadania: opis dziedziny dotyczy wyłącznie elementarnych, natomiast współbicie pojawia się dopiero na poziomie procesów i kwerend.

3.3 Opis poszczególnych zda

Zdanie

A causes α if π

jest regułą efektów. Oznacza ono, że jeżeli akcja prosta A zostaje wykonana w stanie spełniającym warunek π , to każdy dopuszczalny stan wynikowy powinien spełniać formułę α . Reguła ta opisuje zatem bezpośredni, pewny skutek działania i stanowi podstawowy mechanizm reprezentacji zmian zachodzących pod wpływem akcji.

Zdanie

A releases f if π

jest regułą zwolnienia spod działania inercji. Jego sens polega na tym, że wykonanie akcji A w stanie spełniającym π nie wymusza zachowania poprzedniej wartości fluentu f . Fluent ten może zatem po wykonaniu akcji zachować swoją wartość albo ją zmienić. Konstrukcja ta służy do reprezentowania niedeterminizmu lokalnego i pozwala modelować skutki działań, które nie są w pełni określone.

Zdanie

impossible A if π

jest regułą niewykonalności. Oznacza ono, że akcja prosta A nie może zostać wykonana w żadnym stanie spełniającym warunek π . Reguła ta nie opisuje efektu działania, lecz zakazuje samego przejścia. W szczególności podkreślamy, że w klasie $\mathbb{DS4}$ reguły tego rodzaju dotyczą wyłącznie akcji prostych; nie wprowadzamy analogicznych zda dla akcji zoonych na poziomie opisu dziedziny.

Zdanie

always α

jest ograniczeniem integralności. Oznacza ono, że formuła α musi zachodzić w każdym dopuszczalnym stanie systemu. Reguły tego typu nie odnoszą się do pojedynczego działania, lecz do globalnej struktury dziedziny. To wanie one umożliwiają modelowanie ramifikacji, ponieważ wymuszają spójność stanów wynikowych również w zakresie skutków ubocznych.

Zdanie

noninertial f

deklaruje, że fluent f nie podlega prawu inercji. Oznacza to, że jego wartość nie musi być zachowywana przy przejściu między stanami. Konstrukcja ta jest szczególnie użyteczna przy modelowaniu fluentów pochodnych lub zależnych od innych wartości świata, których zmiana nie powinna być traktowana jako naruszenie zasady minimalnej zmiany.

Zdanie

initially α

opisuje wiedzę o stanie początkowym. W najprostszym przypadku może ono wskazywać pojedynczą wartość stanu początkowego, jednak w klasie $\mathbb{DS4}$ dopuszczalny jest opis częściowy. Oznacza to, że zbiór zda tego typu nie musi wyznaczać jednego, w pełni określonego stanu, lecz może pozostawiać niektóre fluenty nieustalone.

Zdanie

α after $A_1, \dots, A_n$

ma charakter specyfikacyjny. Wyraża ono konieczność: po wykonaniu wskazanego ciągu akcji prostych $A_1, \dots, A_n$ formuła α powinna zachodzić zawsze. Zdanie to nie jest lokalną regułą przejścia

dla pojedynczej akcji, lecz warunkiem odnoszącym się do zachowania modelu dla całej sekwencji działań.

Zdanie

observable α after $A_1, \dots, A_n$

równie ma charakter specyfikacyjny. Oznacza ono, że po wykonaniu ciągu akcji prostych $A_1, \dots, A_n$ istnieje co najmniej jeden dopuszczalny przebieg prowadzący do stanu spełniającego α . Podobnie jak zdanie typu **after**, nie opisuje ono bezpośrednio lokalnej dynamiki pojedynczej akcji, lecz własność całego modelu.

3.4 Dziedzina D

Przez dziedzinę działania będziemy rozumieć skończony zbiór zdań języka akcji nad ustalonym sygnaturą $\Upsilon = (\mathcal{F}, \mathcal{A})$. Dziedzinę tak oznaczamy przez D . Zbiór D stanowi pełną formalną specyfikację systemu na poziomie akcji prostych: określa dostępne działania elementarne, ich skutki, warunki niewykonalności, ograniczenia integralności oraz wiedzę o stanie początkowym. W dalszej części opracowania to wanie względem tak określonej dziedziny zostanie zdefiniowana semantyka akcji prostych.

4 Semantyka — poziom akcji prostych

W tej sekcji definiujemy semantykę języka akcji na poziomie akcji prostych. Niech

$$\Upsilon = (\mathcal{F}, \mathcal{A})$$

będzie ustalony sygnatur, gdzie $\mathcal{F}$ jest skończonym zbiorem fluentów, a $\mathcal{A}$ skończonym zbiorem akcji prostych. Niech ponadto D będzie dziedziną działania nad sygnaturą Υ . Przez stan będziemy rozumieć wartościowanie

$$\sigma : \mathcal{F} \rightarrow \{0, 1\},$$

czyli przypisanie każdemu fluentowi wartości logicznej. Dla dowolnej formuły zdaniowej α zapis

$$\sigma \models \alpha$$

oznacza, że formuła α jest prawdziwa w stanie σ .

4.1 Stany dopuszczalne Σ

Nie każdy stan jest stanem legalnym z punktu widzenia opisu dziedziny. Zdania postaci **always** narzucają globalne ograniczenia integralności, które muszą być spełnione w każdym stanie dopuszczalnym. Zbiór stanów dopuszczalnych definiujemy więc następująco:

$$\Sigma = \{\sigma : \forall (\text{always } \alpha) \in D, \sigma \models \alpha\}.$$

Zbiór Σ zawiera dokładnie te stany, które spełniają wszystkie ograniczenia integralności występujące w dziedzinie. Jeżeli pewien stan narusza choć jedno zdanie typu **always**, to nie należy do Σ i nie może wystąpić ani jako stan początkowy, ani jako wynik wykonania jakiejkolwiek akcji. W ten sposób ograniczenia integralności wyznaczają przestrzeń stanów, pomiędzy którymi mogą zachodzić przejścia. Mechanizm ten odpowiada za reprezentację ramifikacji: stan wynikowy musi być nie tylko zgodny z bezpośrednimi efektami działania, lecz również z globalną strukturą dziedziny.

4.2 Stany pocztkowe Σ_0

Zbiór stanów pocztkowych definiujemy jako podzbiór zbioru stanów dopuszczalnych speniających wszystkie zdania pocztkowe:

$$\Sigma_0 = \{\sigma \in \Sigma : \forall(\textbf{initially } \alpha) \in D, \sigma \models \alpha\}.$$

Tak określony zbiór może być jednoelementowy albo wieloelementowy. W szczególności, jeżeli opis pocztkowy nie ustala wartości wszystkich fluentów, to kilka różnych stanów może być zgodnych z wiedzą pocztkową. Przyjęcie zbioru Σ_0 , zamiast pojedynczego wyróżnionego stanu pocztkowego, pozwala modelować czciowy opis stanu pocztkowego bez wprowadzania dodatkowych konstrukcji skadniowych.

Zbiór Σ_0 zależy zatem wyłącznie od zdań typu **initially**; zdania wartościowe (**after**) i obserwacyjne (**observable ... after**) nie wpływają na jego definicję i występują dopiero jako warunki poprawności modelu (M2, §6.4). Konsekwencje oraz uzasadnienie tej konwencji omawiamy w §8.1.

4.3 Funkcja Res_0

Niech $A \in \mathcal{A}$ będzie akcją prostą, a $\sigma \in \Sigma$ stanem dopuszczalnym. Pierwszym krokiem przy wyznaczaniu wyników wykonania akcji A w stanie σ jest określenie wszystkich stanów dopuszczalnych zgodnych z bezpośrednimi efektami aktywnych reguł efektowych. W tym celu definiujemy funkcję pomocniczą

$$\text{Res}_0(A, \sigma) = \{\sigma' \in \Sigma : \forall(A \textbf{ causes } \alpha \textbf{ if } \pi) \in D, \sigma \models \pi \Rightarrow \sigma' \models \alpha\}.$$

Zbiór $\text{Res}_0(A, \sigma)$ zawiera zatem wszystkie stany dopuszczalne, które spełniają efekty wynikające z wykonania akcji A w stanie σ . Definicja ta uwzględnia zarówno skutki bezpośrednie, jak i ograniczenia integralności, ponieważ kandydackie stany wynikowe są wybierane wyłącznie spośród elementów zbioru Σ . Funkcja Res_0 nie rozstrzyga jednak jeszcze, które z tych stanów należy uznać za rzeczywiste wyniki wykonania akcji, ponieważ na tym etapie nie jest jeszcze uwzględniona zasada inercji.

Fluenty inercyjne. Przez $\mathcal{F}_I \subseteq \mathcal{F}$ oznaczamy zbiór *fluentów inercyjnych*, tj. tych, dla których w D nie występuje zdanie **noninertial** f :

$$\mathcal{F}_I = \{f \in \mathcal{F} : (\textbf{noninertial } f) \notin D\}.$$

Fluenty spoza $\mathcal{F}_I$ (pochodne, zależne od innych, „efekty uboczne”) nie podlegają prawu inercji i ich zmiana nie jest uznawana za naruszenie zasady minimalnej zmiany.

Aktywne efekty i zwolnienia. Dla wygody w dalszej części opracowania wprowadzamy skrócone oznaczenia na zbiory aktywnych reguł **causes** oraz **releases** w stanie σ :

$$E(A, \sigma) = \{\alpha : (A \textbf{ causes } \alpha \textbf{ if } \pi) \in D, \sigma \models \pi\},$$

$$R(A, \sigma) = \{f : (A \textbf{ releases } f \textbf{ if } \pi) \in D, \sigma \models \pi\}.$$

Zbiór $E(A, \sigma)$ zawiera wszystkie formuły efektowe, których warunek wstępny jest w σ spełniony; $R(A, \sigma)$ analogicznie dla fluentów zwalnianych spod inercji. Korzystając z tej notacji, zbiór kandydatów wprowadzony wyżej zapisuje się zwięźle jako

$$\text{Res}_0(A, \sigma) = \{\sigma' \in \Sigma : \forall \alpha \in E(A, \sigma), \sigma' \models \alpha\}.$$

Oznaczenia te wykorzystamy zarówno w definicji funkcji New poniżej, jak i w rozdziale o akcjach zoonych (§5).

4.4 Funkcja New

W celu sformalizowania zasady minimalnej zmiany wprowadzamy funkcję

$$\text{New}(A, \sigma, \sigma'),$$

która dla akcji prostej A , stanu wejciowego σ oraz kandydata na stan wynikowy σ' wyznacza zbiór literałów nowych względem stanu wejciowego. Intuicyjnie zbiór ten opisuje zakres zmiany wymaganej do przejścia od σ do σ' .

Formalnie:

$$\text{New}(A, \sigma, \sigma') = \{ l : \sigma' \models l \text{ oraz zachodzi co najmniej jeden z warunków (1)–(2) \},$$

gdzie:

1. $l \in \{f, \neg f\}$ dla pewnego fluentu inercyjnego $f \in \mathcal{F}_I$, oraz $\sigma(f) \neq \sigma'(f)$;
2. $l \in \{f, \neg f\}$ oraz istnieje reguła

$$(A \text{ releases } f \text{ if } \pi) \in D$$

taka, e

$$\sigma \models \pi.$$

Pierwszy warunek odpowiada rzeczywistej zmianie wartości fluentu inercyjnego. Drugi uwzględnia przypadek, w którym aktywna reguła **releases** zwalnia wskazany fluent spod działania inercji. Warto zauważyć, że fluenty zadeklarowane jako **noninertial** nie są traktowane jako zwykłe fluenty inercyjne, a zatem ich zmiana nie zwiększa kosztu przejścia w sensie zasady minimalnej zmiany.

4.5 Minimalizacja — $\text{Res}(A, \sigma)$

Wacny zbiór wyników wykonania akcji prostej A w stanie σ otrzymujemy przez wybór tych stanów z $\text{Res}_0(A, \sigma)$, dla których zbiór $\text{New}(A, \sigma, \sigma')$ jest minimalny względem inkluzji. Definiujemy więc:

$$\text{Res}(A, \sigma) = \{ \sigma' \in \text{Res}_0(A, \sigma) : \nexists \sigma'' \in \text{Res}_0(A, \sigma) (\text{New}(A, \sigma, \sigma'') \subset \text{New}(A, \sigma, \sigma')) \}.$$

Zbiór $\text{Res}(A, \sigma)$ zawiera dokładnie te stany, które spełniają wszystkie aktywne efekty bezporównania i jednocześnie wprowadzają najmniejszą konieczną zmianę względem stanu wejciowego. Minimalność rozumiana jest tutaj względem inkluzji zbiorów, a nie względem ich licznoci. Dzięki temu zasada inercji przyjmuje postać preferencji dla tych stanów wynikowych, które zachowują jak najwięcej własności stanu poprzedniego, o ile nie stoi to w sprzeczności z efektami akcji ani z ograniczeniami integralności.

Jeżeli kilka stanów osiąga ten sam stopień minimalności, wszystkie pozostają dopuszczalnymi wynikami wykonania akcji. Niedeterminizm może więc wynikać zarówno z obecności reguły **releases**, jak i z wielości różnych minimalnych stanów zgodnych z ograniczeniami dziedziny.

4.6 Niewykonalność

Osobnego traktowania wymaga przypadek niewykonalności akcji. Jeżeli istnieje reguła

$$(\text{impossible } A \text{ if } \pi) \in D$$

taka, e

$$\sigma \models \pi,$$

to przyjmujemy

$$\text{Res}(A, \sigma) = \emptyset.$$

Oznacza to, e akcja A nie ma w stanie σ adnego dopuszczalnego wyniku, poniewa nie moe zosta wykonana. Warunek niewykonalnoci ma zatem charakter priorytetowy: jeli akcja jest zabroniona, nie analizuje si jej efektów, lecz od razu uznaje si, e zbiór wyników jest pusty. Ujcie to odpowiada interpretacji reguy **impossible** jako szczególnego przypadku reguy efektowej prowadzcej do sprzeczności, a zarazem zapewnia przejrzysto dalszych definicji dotyczcych wykonalności procesów.

Podsumowujc, semantyka akcji prostych opiera si na nastpujcm schemacie: najpierw wyznaczany jest zbiór stanów dopuszczalnych Σ , nastpnie zbiór stanów pocztkowych Σ_0 , dalej dla kadej akcji prostej i kadeego stanu dopuszczalnego budowany jest zbiór kandydatów $\text{Res}_0(A, \sigma)$, a ostatecznie sporód tych kandydatów wybierane s stany minimalne wzgldem funkcji New. Tak zdefiniowana semantyka czy bezporednie efekty dziaa, ograniczenia integralności, zasad inercji, niedeterminizm oraz niewykonalno w jednolity model przejd dla akcji prostych.

5 Semantyka — akcje zoone

W niniejszym rozdziale rozszerzamy semantyk wprowadzon w §4 na poziom akcji zoonych, rozumianych jako zbiory akcji prostych wykonywanych równoczenie. Poniewa w klasie $\mathbb{DS4}$ zdania dziedziny odnosz si wycznie do akcji prostych, akcje zoone pojawiaj si dopiero na poziomie procesów i s konstruowane dynamicznie. Podstaw teoretyczn stanowi jzyk AC ; ponisze definicje adaptuj mechanizmy detekcji konfliktów i dekompozycji znane z tego jzyka do warunków zadania.

5.1 Akcja zoona $\mathbb{A}$

Akcj zoon nad sygnatur $\Upsilon = (\mathcal{F}, \mathcal{A})$ nazywamy dowolny niepusty zbiór akcji prostych

$$\mathbb{A} = \{A_1, \dots, A_k\} \subseteq \mathcal{A}, \quad \mathbb{A} \neq \emptyset.$$

Intuicyjnie, skadowe akcji zoonej maj zosta wykonane równoczenie w tym samym stanie. Zbiór wszystkich akcji zoonych oznaczamy przez

$$\mathcal{A}_z = 2^{\mathcal{A}} \setminus \{\emptyset\}.$$

Zbiór ten stanowi alfabet procesów w jzyku kwerend (§6.1), gdzie kady krok procesu jest elementem $\mathcal{A}_z$.

Dla wygody notacyjnej przyjmujemy konwencj, zgodnie z któr pojedyncza akcja prosta $A \in \mathcal{A}$ wystpujca w procesie jest skrótem oznaczajcym singleton $\{A\} \in \mathcal{A}_z$. Dziki temu semantyka akcji zoonych, wprowadzona poniej, obejmuje jako przypadek szczególny semantyk akcji prostej z rozdziau 4.5: dla $\mathbb{A} = \{A\}$ konstrukcje z niniejszej sekcji redukuj si do tych z §4.

Podkrełmy, e w klasie $\mathbb{DS4}$ akcje zoone nie s elementami sygnatury ani skadni opisu dziedziny. Oznacza to w szczególności, e dla akcji zoonej nie mona zadeklarowa reguy **causes**, **releases** ani **impossible**; jej zachowanie jest w peni wyznaczone przez zachowanie skadowych akcji prostych oraz przez mechanizm dekompozycji opisany dalej.

5.2 Konflikty

Przy równoczesnym wykonaniu kilku akcji prostych moe doj do sytuacji, w której ich skutki s ze sob niezgodne — na przykad jedna akcja wymusza zajcie literau f , a druga literau $\neg f$. W takim przypadku akcje te nazywamy *kolidujcymi* w danym stanie. Definicj konfliktu przyjmujemy za jzykiem AC i formujemy j zawsze wzgldem ustalonego stanu $\sigma \in \Sigma$, poniewa zestaw aktywnych regu **causes** oraz **releases** zaley od tego, które warunki wstpne π s w σ spenione.

Przypominamy oznaczenia wprowadzone w §4: dla akcji prostej $A \in \mathcal{A}$ i stanu $\sigma \in \Sigma$

$$E(A, \sigma) = \{\alpha : (A \textbf{ causes } \alpha \textbf{ if } \pi) \in D, \sigma \models \pi\}$$

oznacza zbiór formu efektowych aktywnych w σ , a $R(A, \sigma) = \{f : (A \textbf{ releases } f \textbf{ if } \pi) \in D, \sigma \models \pi\}$ — zbiór fluentów zwalnianych spod inercji.

Niech $A, B \in \mathcal{A}$ bd dwiema różnymi akcjami prostymi, a $\sigma \in \Sigma$ ustalonym stanem dopuszczalnym. Mówimy, e akcje A i B **koliduj w stanie** σ , jeeli zachodzi co najmniej jeden z dwóch warunków:

Przypadek 1 (*causes* vs. *causes*). ¹ Istnieją formuły $\alpha \in E(A, \sigma)$ oraz $\beta \in E(B, \sigma)$ takie, e w postaciach normalnych

$$\text{DNF}(\alpha) = C_1^\alpha \vee \dots \vee C_p^\alpha, \quad \text{DNF}(\beta) = C_1^\beta \vee \dots \vee C_q^\beta$$

występuje para skadników C_i^α, C_j^β , w której pewien litera z C_i^α jest komplementarny względem pewnego literau z C_j^β , tj. istnieje fluent $f \in \mathcal{F}$ taki, e $f \in C_i^\alpha$ i $\neg f \in C_j^\beta$, albo $\neg f \in C_i^\alpha$ i $f \in C_j^\beta$.

Przypadek 2 (*causes* vs. *releases*). Istnieje formuła $\alpha \in E(A, \sigma)$ oraz fluent $f \in R(B, \sigma)$ takie, e f lub $\neg f$ występuje jako litera w pewnym skadniku postaci $\text{DNF}(\alpha)$. Symetrycznie dopuszczamy tę sytuację po zamianie ról A i B , tj. $\alpha \in E(B, \sigma)$ oraz $f \in R(A, \sigma)$.

Intuicyjnie, Przypadek 1 opisuje sytuację, w której obie akcje wymuszają bezporednio sprzeczne wartoci tego samego fluentu, natomiast Przypadek 2 odnosi się do konfliktu między akcją wymuszającą warto fluentu i akcją, która ten sam fluent jednoczenie zwalnia spod działania inercji. Posuwanie się postaci DNF jest tutaj istotne: formuły efektowe mogą być dowolnie złożonymi formułami zdaniowymi, a badanie zgodności literałów bezporednio w postaci DNF pozwala jednolicie rozpatrywać wszystkie alternatywy sposobu realizacji danego efektu.

Z symetrii definicji wynika, e relacja konfliktu w stanie σ jest symetryczna. Możemy zatem zakodować ją jako nieskierowany graf konfliktu

$$G_\sigma(\mathbb{A}) = (V, E), \quad V = \mathbb{A}, \quad E = \{\{A_i, A_j\} : A_i, A_j \in \mathbb{A}, A_i \neq A_j, A_i \text{ i } A_j \text{ kolidują w } \sigma\}.$$

Graf ten wykorzystamy w §5.3 do charakteryzacji dekompozycji oraz w pseudokodzie w §9.4 (algorytm 4).

5.3 Dekompozycje $\Delta(\mathbb{A}, \sigma)$

Jeeli pewne skadowe akcji zoonej $\mathbb{A}$ kolidują w stanie σ , nie możemy wykonać wszystkich z nich równocześnie. Zamiast całkowicie blokować taką akcję, za językiem AC dopuszczamy wykonanie dowolnego maksymalnego podzbioru skadowych, który można bezpiecznie uruchomić razem. Taki podzbiór nazywamy dekompozycją.

Najpierw wyróżnimy akcje, które w ogóle mogą zostać wykonane pojedynczo w rozważanym stanie. Mówimy, e akcja prosta $A \in \mathcal{A}$ jest **indywidualnie wykonalna** w stanie σ wtedy i tylko wtedy, gdy $\text{Res}(A, \sigma) \neq \emptyset$. W szczególności każda akcja, dla której w stanie σ aktywne jest zdanie **impossible**, nie jest indywidually wykonalna (por. §4.6). Zbiór skadowych indywidually wykonalnych oznaczamy przez

$$V_{\text{exec}}(\mathbb{A}, \sigma) = \{A \in \mathbb{A} : \text{Res}(A, \sigma) \neq \emptyset\}.$$

Definicja (dekompozycja). Podzbiór $\delta \subseteq \mathbb{A}$ jest **dekompozycją** akcji zoonej $\mathbb{A}$ w stanie σ , jeeli spełnia trzy warunki:

¹W wykładowej prezentacji języka AC przypadki s ponumerowane odwrotnie (konflikt *causes/releases* jako 1, *causes/causes* jako 2). Zachowujemy numerację z checkpointu 1, traktując prostszy (i częściej spotykany) przypadek *causes/causes* jako pierwszy.

(D1) Lokalna wykonalno. Kada akcja $A \in \delta$ jest indywidualnie wykonalna w σ , tj. $\delta \subseteq V_{\text{exec}}(\mathbb{A}, \sigma)$.

(D2) Bezkonfliktowo. adna para rónych akcji $A, B \in \delta$ nie koliduje w σ . Równowanie, δ jest zbiorem niezaleny w grafie konfliktu $G_\sigma(\mathbb{A})$.

(D3) Maksymalno. Nie istnieje akcja $A \in V_{\text{exec}}(\mathbb{A}, \sigma) \setminus \delta$, dla której $\delta \cup \{A\}$ nadal spenia (D1) i (D2).

Zbiór wszystkich dekompozycji akcji $\mathbb{A}$ w stanie σ oznaczamy przez $\Delta(\mathbb{A}, \sigma)$. Maksymalno rozumiana jest względm inkluzji, a nie względm licznoci, co oznacza, e zbiór $\Delta(\mathbb{A}, \sigma)$ moe zawiera dekompozycje o rónej liczbie elementów.

Lemat (istnienie dekompozycji). Jeeli $V_{\text{exec}}(\mathbb{A}, \sigma) \neq \emptyset$, to $\Delta(\mathbb{A}, \sigma) \neq \emptyset$.

Szkic dowodu. Niech $A_0 \in V_{\text{exec}}(\mathbb{A}, \sigma)$. Zbiór jednoelementowy $\delta_0 = \{A_0\}$ trywialnie spenia warunek (D1) oraz, z uwagi na brak par rónych akcji, równie warunek (D2). Konstruujemy cig zbiorów $\delta_0 \subseteq \delta_1 \subseteq \dots$ nastpujco: dopóki istnieje akcja $A \in V_{\text{exec}}(\mathbb{A}, \sigma) \setminus \delta_i$ taka, e $\delta_i \cup \{A\}$ wci spenia (D1) oraz (D2), kadziemy $\delta_{i+1} = \delta_i \cup \{A\}$. Poniewa $\mathbb{A}$ jest zbiorem skoczonym, procedura zatrzymuje si po skoczonych liczbie kroków na pewnym zbiorze δ^* . Z konstrukcji δ^* spenia (D1) i (D2), a jego maksymalno wynika z warunku zatrzymania — adnej akcji nie mona ju doda bez naruszenia (D1) lub (D2). Zatem $\delta^* \in \Delta(\mathbb{A}, \sigma)$. $\square$

Przypadek graniczny. Jeeli natomiast $V_{\text{exec}}(\mathbb{A}, \sigma) = \emptyset$, tj. adna skadowa akcji $\mathbb{A}$ nie jest w stanie σ indywidualnie wykonalna, to nie istnieje adna dekompozycja speniajca (D1), a wic

$$\Delta(\mathbb{A}, \sigma) = \emptyset \quad \text{oraz} \quad \text{Res}(\mathbb{A}, \sigma) = \emptyset.$$

Uzasadnienie takiej konwencji oraz jej interpretacja znajduj si w §8.3; tutaj traktujemy j jako integraln cz definicji.

5.4 Res($\mathbb{A}, \sigma$) przez dekompozycje

Majc wyznaczony zbiór dekompozycji $\Delta(\mathbb{A}, \sigma)$, definiujemy wynik wykonania akcji zoonej w trzech etapach: najpierw lokalnie, dla pojedynczej dekompozycji δ , analogicznie jak dla akcji prostej z §4, a nastpnie przez sumowanie wyników po wszystkich dekompozycjach.

Efekty i zwolnienia dekompozycji. Dla dekompozycji $\delta \in \Delta(\mathbb{A}, \sigma)$ definiujemy zbiorcze zestawy aktywnych efektów oraz aktywnych zwolnie:

$$E(\delta, \sigma) = \bigcup_{A \in \delta} E(A, \sigma), \quad R(\delta, \sigma) = \bigcup_{A \in \delta} R(A, \sigma).$$

Poniewa w δ nie wystpuj pary kolidujce, suma $E(\delta, \sigma)$ jest (jako zbiór formu do jednoczesnego spienienia) niesprzeczna — to wanie gwarantuje bezkonfliktowo skadowych.

Kandydaci i zbiór zmian. Definiujemy zbiór stanów dopuszczalnych zgodnych z efektami dekompozycji:

$$\text{Res}_0(\delta, \sigma) = \{\sigma' \in \Sigma : \forall \alpha \in E(\delta, \sigma), \sigma' \models \alpha\}.$$

Dla kadego kandydata $\sigma' \in \text{Res}_0(\delta, \sigma)$ wyznaczamy zbiór literaów nowych, analogicznie jak w §4.4:

$$\text{New}(\delta, \sigma, \sigma') = \{l : \sigma' \models l \text{ oraz zachodzi co najmniej jeden z warunków (1)–(2)}\},$$

gdzie (1) $l \in \{f, \neg f\}$ dla pewnego fluentu inercyjnego $f \in \mathcal{F}_I$, oraz $\sigma(f) \neq \sigma'(f)$; (2) istnieje fluent $f \in R(\delta, \sigma)$ taki, e $l \in \{f, \neg f\}$. Tak jak dla akcji prostej, fluenty zadeklarowane jako **noninertial** nie s zaliczane do zbioru zmian.

Minimalizacja lokalna. Wynikiem dekompozycji δ w stanie σ jest podzbiór kandydatów minimalnych względem zawierania zbioru New:

$$\text{Res}(\delta, \sigma) = \{\sigma' \in \text{Res}_0(\delta, \sigma) : \nexists \sigma'' \in \text{Res}_0(\delta, \sigma) (\text{New}(\delta, \sigma, \sigma'') \subset \text{New}(\delta, \sigma, \sigma'))\}.$$

Wynik akcji zoonej. Ostateczny zbiór stanów wynikowych akcji zoonej $\mathbb{A}$ w stanie σ powstaje jako suma wyników po wszystkich dekompozycjach:

$$\text{Res}(\mathbb{A}, \sigma) = \bigcup_{\delta \in \Delta(\mathbb{A}, \sigma)} \text{Res}(\delta, \sigma). \quad (1)$$

Na poziomie akcji zoonej rezygnujemy z jednolitej funkcji $\text{New}(\mathbb{A}, \sigma, \sigma')$: minimalizacja odbywa się lokalnie wewnątrz każdej dekompozycji (krok „Minimalizacja lokalna” powyżej), a końcowy zbiór $\text{Res}(\mathbb{A}, \sigma)$ powstaje jako suma wyników lokalnych. Taka konstrukcja jest zgodna z warunkiem (M3b) modelu dziedziny (§6.4): stany wynikowe akcji zoonej s to dokładnie te stany dopuszczalne, które speniają czne efekty pewnej dekompozycji i wprowadzają najmniej koniecznych zmian w obrębie tej dekompozycji.

Niedeterminizm. Istnienie wielu dekompozycji w tym samym stanie stanowi nowe, niezależne źródło niedeterminizmu w klasie $\mathbb{DS4}$ — obok niedeterminizmu pochodzącego od regu **releases** i od wielości stanów minimalnych. W przypadku granicznym, gdy jedna para składowych nie koliduje, istnieje dokładnie jedna dekompozycja $\delta = V_{\text{exec}}(\mathbb{A}, \sigma)$, a konstrukcja (1) redukuje się do obliczenia pojedynczego wyniku analogicznego do akcji prostej.

5.5 Uwaga o „restricted inheritance”

W penym języku AC dziedzina dopuszcza zdania dotyczące bezpośrednio akcji zoonych, na przykład zdania typu $\mathbb{A}$ **causes** α **if** π dla $\mathbb{A}$ będącego zbiorem akcji. W takim przypadku efekt ogłoszony dla $\mathbb{A}$ nie musi być w ogólności dziedziczony przez wszystkie jej podzbiory — mechanizm ten nosi w literaturze nazwę *restricted inheritance*.

W klasie $\mathbb{DS4}$ opisy dziedziny zawierają wyłącznie zdania odnoszące się do akcji prostych (patrz §3); żadne zdanie nie deklaruje efektu zbioru akcji. Z tego powodu problem ograniczonego dziedziczenia nie występuje: efekty każdej akcji prostej A_i są dziedziczone w całości przez każdą dekompozycję zawierającą A_i , a czyny efekt dekompozycji powstaje jako suma zbiorów $E(A_i, \sigma)$ oraz $R(A_i, \sigma)$ zgodnie z §5.4. Stanowi to wiadome uproszczenie mechanizmu AC dopuszczalne w zakresie zadania projektowego.

5.6 Porównanie z trzecim projektem

Tre zadania projektowego proponuje prostszy mechanizm obsługi współbieżności: akcje zoone wykonują się poprawnie, jeżeli ich składowe operują na rozłącznych zbiorach fluentów. Poniżej pokazujemy, że nasz mechanizm detekcji konfliktów zaczerpnięty z AC jest formalnie silniejszy — warunek rozłączności jest jego szczególnym przypadkiem. Merytoryczne uzasadnienie wyboru ogólniejszego mechanizmu znajduje się w §8.2; tutaj ograniczamy się do argumentu formalnego.

Dla akcji prostej $A \in \mathcal{A}$ oraz stanu $\sigma \in \Sigma$ definiujemy zbiór fluentów, na które A *potencjalnie wpływa* w stanie σ :

$$\text{Vars}(A, \sigma) = \left(\bigcup_{\alpha \in E(A, \sigma)} \text{fl}(\alpha) \right) \cup R(A, \sigma),$$

gdzie $\text{fl}(\alpha)$ oznacza zbiór fluentów występujących w formule α (niezależnie od polaryzacji literału). Zbiór $\text{Vars}(A, \sigma)$ obejmuje zatem wszystkie fluenty, które mogą zmienić swoją wartość pod wpływem aktywnych regu **causes** lub **releases** dla akcji A .

Twierdzenie (rozczo implikuje brak konfliktu). Niech $A, B \in \mathcal{A}$ bd rónymi akcjami prostymi i niech $\sigma \in \Sigma$ bdzie dowolnym stanem dopuszczalnym. Jeeli $\text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma) = \emptyset$, to akcje A i B nie koliduj w σ w sensie §5.2.

Dowód. Zaómy nie wprost, e akcje A i B koliduj w σ . Rozwaymy oba przypadki definicji konfliktu.

Przypadek 1. Istniej $\alpha \in E(A, \sigma)$, $\beta \in E(B, \sigma)$ oraz fluent f taki, e f lub $\neg f$ wystpuje w pewnym skadniku $\text{DNF}(\alpha)$, a komplementarny litera — w pewnym skadniku $\text{DNF}(\beta)$. W szczególności fluent f pojawia si w formule α oraz w formule β , zatem $f \in \text{fl}(\alpha) \subseteq \text{Vars}(A, \sigma)$ oraz $f \in \text{fl}(\beta) \subseteq \text{Vars}(B, \sigma)$. Std $f \in \text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma)$ — sprzeczno z zaóeniem rozczo.

Przypadek 2. Istnieje (bez utraty ogólnoci) formua $\alpha \in E(A, \sigma)$ oraz fluent $f \in R(B, \sigma)$ taki, e f lub $\neg f$ wystpuje w pewnym skadniku $\text{DNF}(\alpha)$. Wówczas $f \in \text{fl}(\alpha) \subseteq \text{Vars}(A, \sigma)$ oraz $f \in R(B, \sigma) \subseteq \text{Vars}(B, \sigma)$, co daje $f \in \text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma)$ — ponownie sprzeczno.

W obu przypadkach otrzymalimy sprzeczno, wic akcje A i B nie koliduj w σ . $\square$

Wniosek. Jeeli wszystkie pary skadowych akcji zoonej $\mathbb{A}$ operuj na parami rocznych zbiorach fluentów, to graf konfliktu $G_\sigma(\mathbb{A})$ nie zawiera adnej krawdzi, a zatem $\Delta(\mathbb{A}, \sigma) = \{V_{\text{exec}}(\mathbb{A}, \sigma)\}$. W tej sytuacji mechanizm AC i warunek rozczo z treci zadania prowadzi do identycznych wyników. Przykad w §10.2 (producent i konsumenci) pokazuje natomiast, e odwrotna implikacja nie zachodzi: para akcji operujca na czciowo wspólnym zbiorze fluentów moe pozosta bezkonfliktowa, o ile nie prowadzi do komplementarnych literaów w sensie Przypadku 1 ani do sytuacji z Przypadku 2.

6 Procesy i model dziedziny

6.1 Proces $\mathbb{P}$

Proces $\mathbb{P} = (\mathbb{A}_1, \dots, \mathbb{A}_n)$, $n \geq 0$, to cig akcji zoonych wykonywanych jedna po drugiej. Dla $n = 0$ otrzymujemy *proces pusty*, oznaczany przez ε ; dla $n \geq 1$ proces modeluje uporzdkowany cig akcji nastpujcych po sobie, z których kada moe by akcj zoon reprezentujc zdarzenia równolege.

Przez $\mathcal{P} = \mathcal{A}_z^*$ oznaczamy zbiór wszystkich procesów (domknięcie Kleene’ego nad alfabetem $\mathcal{A}_z$), co obejmuje równie proces pusty ε odpowiadajcy $n = 0$.

6.2 Funkcja Ψ

Niech $\mathcal{F}_S$ oznacza rodzin wszystkich czciowych funkcji

$$\Psi_S : \mathcal{P} \times \Sigma \rightarrow \Sigma,$$

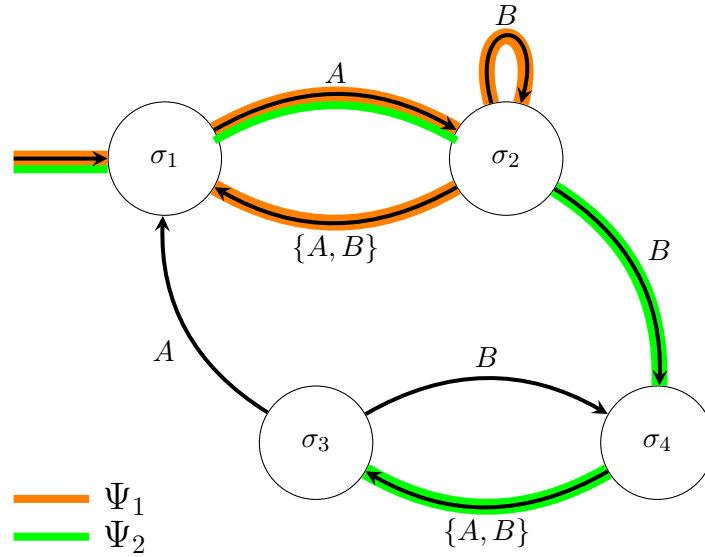
speniajcych nastpujcy warunek rekurencyjny:

- (baza) $\Psi_S(\varepsilon, \sigma) = \sigma$ dla kadego $\sigma \in \Sigma$;
- (krok) dla procesu $\mathbb{P}_n = (\mathbb{A}_1, \dots, \mathbb{A}_n)$, $n \geq 1$, warto $\Psi_S(\mathbb{P}_n, \sigma)$ jest zdefiniowana wtedy i tylko wtedy, gdy

1. $\Psi_S(\mathbb{P}_{n-1}, \sigma)$ jest zdefiniowane; oznaczmy $\sigma' = \Psi_S(\mathbb{P}_{n-1}, \sigma)$,
2. $\text{Res}(\mathbb{A}_n, \sigma') \neq \emptyset$;

wówczas $\Psi_S(\mathbb{P}_n, \sigma) \in \text{Res}(\mathbb{A}_n, \sigma')$.

Kady element $\Psi_S \in \mathcal{F}_S$ odpowiada pojedynczemu *ladowi wykonania*: dokonuje deterministycznego wyboru poród stanów osiganych w kadym niedeterministycznym kroku. Obrazuje to rysunek 1, na którym kolory odpowiadaj dwóm rónym elementom $\mathcal{F}_S$ dla tego samego procesu $\mathbb{P}$. W dalszych czciach opracowania kwantyfikatory „dla kadego Ψ_S ” oraz „istnieje Ψ_S ” (§6.3, §7.3) interpretujemy jako kwantyfikatory po rodzinie $\mathcal{F}_S$.



Rysunek 1: Przykładowe funkcje Ψ dla procesu $\mathbb{P} = (A, B, \{A, B\})$

6.3 Prawdziwość wartościowa

Zdania wartościowe są prawdziwe w $S = (\Sigma, \Sigma_0, \text{Res})$ jeśli zachodzą warunki:

α after $A_1, A_2, \dots, A_n$

Dla każdego $\sigma_0 \in \Sigma_0$ i dla każdej funkcji $\Psi_S \in \mathcal{F}_S$: jeśli $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0)$ jest zdefiniowana, to $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0) \models \alpha$.

observable α after $A_1, A_2, \dots, A_n$

Istnieje $\sigma_0 \in \Sigma_0$ oraz funkcja $\Psi_S \in \mathcal{F}_S$ takie, że $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0)$ jest zdefiniowana oraz $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0) \models \alpha$.

Uwaga: after w dziedzinie vs. after w kwerendach. Zdanie wartościowe α after $A_1, \dots, A_n$ występuje w dziedzinie D traktujemy — zgodnie z klasycznym ujęciem AR — jako warunek poprawności modelu: jeśli pewien łańcuch kończy się w stanie niezdefiniowanym (proces się nie kończy), to implikacja „jeśli zdefiniowana, to $\models \alpha$ ” jest spełniona pusto i zdanie nadal może być prawdziwe. Inaczej zachowuje się analogicznie wygładzająca *kwerenda necessary α after $\mathbb{P}$* (§7.3), która wymaga, by $\Psi_S(\mathbb{P}, \sigma_0)$ była zdefiniowana i spełniała α — niewykonalny proces zawsze daje odpowiedź fałszywą. Rozróżnienie to podsumowujemy w §8.4.

6.4 Model dziedziny

Struktura $S = (\Sigma, \Sigma_0, \text{Res})$ jest modelem dziedziny D , jeśli:

- (M1) Σ jest zbiorem wszystkich stanów dopuszczalnych w D (patrz §4.1); zbiór Σ_0 jest wyznaczony przez zdania **initially** (patrz §4.2).
- (M2) Każde zdanie wartościowe i obserwacyjne (**after** oraz **observable ... after**) występuje w D jest prawdziwe w S (patrz §6.3).
- (M3a) Dla każdej akcji prostej $A \in \mathcal{A}$ i każdego stanu $\sigma \in \Sigma$:

$$\text{Res}(A, \sigma) = \{\sigma' \in \text{Res}_0(A, \sigma) : \text{New}(A, \sigma, \sigma') \text{ jest minimalny w.r.t. inkluzji}\}$$

(patrz §4.5).

(M3b) Dla kadej akcji zoonej $\mathbb{A} \in \mathcal{A}_z$ i kadego stanu $\sigma \in \Sigma$:

$$\text{Res}(\mathbb{A}, \sigma) = \bigcup_{\delta \in \Delta(\mathbb{A}, \sigma)} \text{Res}(\delta, \sigma),$$

przy czym kady $\text{Res}(\delta, \sigma)$ jest obliczany analogicznie jak w (M3a), wzgldem $\text{Res}_0(\delta, \sigma)$ oraz $\text{New}(\delta, \sigma, \cdot)$ (patrz §5.4).

Warunek **M1** wymusza zachowanie zda integralnociowych **always** oraz **initially**. Warunek **M2** — zachowanie zda wartociowych i obserwacyjnych. Warunki **M3a/M3b** wi Res z dziedzin (**causes**, **releases**, **impossible**) zgodnie z zasad minimalnej zmiany, najpierw na poziomie akcji prostej, a nastpnie przez sumowanie wyników lokalnie minimalnych dla wszystkich dekompozycji akcji zoonej.

Definicja modelu jest inspirowana semantyk języka AC i stanowi jej adaptacj do klasy DS4 (opisy dziedzin ograniczone do akcji prostych, dynamiczna detekcja konfliktów, bez mechanizmu *restricted inheritance* — patrz §5.5, §5.6).

6.5 Jednoznaczno modelu

Warunki (M1) i (M3a)/(M3b) wyznaczaj kandydujc struktur $S^* = (\Sigma, \Sigma_0, \text{Res})$ w sposób jednoznaczny — wszystkie jej elementy s bowiem w peni okrelone przez sam dziedzin:

- Σ przez zdania **always** (§4.1);
- Σ_0 przez zdania **initially** (§4.2, §8.1);
- Res przez zdania **causes**, **releases** oraz **impossible** wraz z minimalizacj lokaln (§4.5, §5.4).

Specyfika języka wymaga niedeterminizmu — pojedyncza akcja moe przeprowadza stan do kilku różnych stanów; w takich sytuacjach Res jest zbiorem *wszystkich* moliwych stanów docelowych, wiec jest wyznaczony jednoznacznie.

Ta kandydujca struktura S^* jest modelem dziedziny D wtedy i tylko wtedy, gdy dodatkowo spenia warunek (M2), tzn. kade zdanie typu **after** i **observable** ... **after** z D zachodzi w S^* . Jeeli cho jedno z tych zda jest naruszone w S^* , dziedzina *nie ma* modelu; nie stanowi to sprzeczności w definicji, lecz wasno samej dziedziny. Operacyjnie nie jest konieczne sprawdzanie (M2) na caym S^* z góry: weryfikacja odbywa si punktowo, przy okazji ewaluacji kwerend (§9.7).

Konsekwencj jednoznaczności kandydata S^* jest to, e kwantyfikator po modelach w definicji relacji $D \models Q$ (§7.3) sprowadza si do sprawdzenia Q w tym jednym S^* , przy czym warunek (M2) traktujemy jako dodatkow hipotez modelu; dla dziedziny bez modelu $D \models Q$ zachodzi trywialnie dla kadego Q .

7 Jzyk kwerend

7.1 Zdania języka kwerend

Jzyk kwerend zawiera cztery moliwe zapytania. Dwie pierwsze dotycz osigania stanu po wykonaniu procesu. Dwie nastpne odpowiadaj na pytanie, czy proces jest wykonalny. W obu przypadkach kwerendy wystpuj w wersjach *necessary* i *possibly*: wersja *necessary* wymaga, by warunek zachodzi dla *kadego* stanu pocztkowego $\sigma_0 \in \Sigma_0$ oraz *kadego* ladu wykonania Ψ , natomiast wersja *possibly* wymaga, by warunek zachodzi dla *pewnego* stanu pocztkowego i *pewnego* ladu wykonania. Zestawienie wszystkich dopuszczalnych konstrukcji przedstawia tabela 3.

Tabela 3: Typy zda języka kwerend dla klasy $\mathbb{DS4}$

Zdanie (zapis formalny)	Znaczenie intuicyjne
necessary α after $\mathbb{P}$	Czy po wykonaniu procesu $\mathbb{P}$ z kadego stanu pocztkowego, na kadym ladzie wykonania, formua α zachodzi?
possibly α after $\mathbb{P}$	Czy istnieje stan pocztkowy i lad wykonania procesu $\mathbb{P}$ taki, e po jego wykonaniu formua α zachodzi?
necessary executable $\mathbb{P}$	Czy wykonanie procesu $\mathbb{P}$ jest moliwe z kadego stanu pocztkowego na kadym ladzie wykonania?
possibly executable $\mathbb{P}$	Czy istnieje stan pocztkowy i lad wykonania, na którym proces $\mathbb{P}$ jest wykonalny?

7.2 Skadnia

W języku kwerend wystpuj procesy, bdce cigami akcji zoonych. Skadni dla tych struktur definiujemy nastpujco:

$$\begin{aligned}
\langle \text{proces} \rangle &::= "(" \langle \text{proces-wnetrze} \rangle ")" \\
\langle \text{proces-wnetrze} \rangle &::= \langle \text{proces-wnetrze} \rangle ", " \langle \text{akcja-zlozona} \rangle | \langle \text{akcja-zlozona} \rangle \\
\langle \text{akcja-zlozona} \rangle &::= "{" \langle \text{akcja-zlozona-wnetrze} \rangle "}" \\
\langle \text{akcja-zlozona-wnetrze} \rangle &::= \langle \text{akcja-zlozona-wnetrze} \rangle ", " \langle \text{akcja} \rangle | \langle \text{akcja} \rangle
\end{aligned}$$

Waciw skadni języka kwerend definiujemy nastpujco.

$$\begin{aligned}
\langle \text{kw-wartosciowa-nec} \rangle &::= \text{"necessary"} \langle \text{formula} \rangle \text{"after"} \langle \text{proces} \rangle \\
\langle \text{kw-wartosciowa-pos} \rangle &::= \text{"possibly"} \langle \text{formula} \rangle \text{"after"} \langle \text{proces} \rangle \\
\langle \text{kw-wykonalnosci-nec} \rangle &::= \text{"necessary executable"} \langle \text{proces} \rangle \\
\langle \text{kw-wykonalnosci-pos} \rangle &::= \text{"possibly executable"} \langle \text{proces} \rangle
\end{aligned}$$

7.3 Semantyka formalna

Relacja $D \approx Q$

Kwerenda Q jest konsekwencj dziedziny D , jeli zachodzi w kadym jej modelu S . Poniewa model dziedziny w klasie $\mathbb{DS4}$ jest co najwyzej jeden (6.5), wystarczy sprawdzi, czy Q zachodzi w tym (jedynym, o ile istnieje) modelu. Jeli model nie istnieje, relacja $D \approx Q$ zachodzi trywialnie dla kadej kwerendy Q (pusty kwantyfikator po modelach).

Kwerendy dla wielu stanów pocztkowych

Z zaoe problemu model moe mie wiele rónych stanów pocztkowych $\sigma_0 \in \Sigma_0$, a z powodu nie-determinizmu Res kadý taki stan moe generowa wiele rónych ladów wykonania Ψ_S . Kwerend

necessary uznajemy za prawdziw, gdy warunek kwerendy jest speniony dla *każdego* stanu pocztkowego oraz *każdego* ladu wykonania. Kwerend **possibly** uznajemy za prawdziw, gdy warunek kwerendy jest speniony dla *pewnego* stanu pocztkowego i *pewnego* ladu wykonania.

Definicja spenialnoci kwerend

Q : **necessary α after $\mathbb{P}$**

$D \models Q$ wtw., gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ dla każdego stanu pocztkowego $\sigma_0 \in \Sigma_0$ oraz dla każej funkcji $\Psi_S \in \mathcal{F}_S$ warto $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowana i $\Psi_S(\mathbb{P}, \sigma_0) \models \alpha$.

Q : **possibly α after $\mathbb{P}$**

$D \models Q$ wtw., gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ istnieje stan pocztkowy $\sigma_0 \in \Sigma_0$ oraz funkcja $\Psi_S \in \mathcal{F}_S$ takie, e $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowana i $\Psi_S(\mathbb{P}, \sigma_0) \models \alpha$.

Q : **necessary executable $\mathbb{P}$**

$D \models Q$ wtw., gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ dla każdego stanu pocztkowego $\sigma_0 \in \Sigma_0$ oraz dla każej funkcji $\Psi_S \in \mathcal{F}_S$ warto $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowana.

Q : **possibly executable $\mathbb{P}$**

$D \models Q$ wtw., gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ istnieje stan pocztkowy $\sigma_0 \in \Sigma_0$ oraz funkcja $\Psi_S \in \mathcal{F}_S$ takie, e $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowana.

8 Decyzje projektowe i uzasadnienia

Poniej zbieramy najwaniemjsze decyzje projektowe podjte przy opracowywaniu klasy DS4 — zarówno miejsca, w których wiadomie odchodzimy od trci zadania lub standardowej postaci jzyków AR/AC , jak i rozstrzygnięcia interpretacyjne przypadków brzegowych. Szczegółowe uzasadnienia znajduj si w odpowiednich sekcjach formalnych; tutaj prezentujemy je w skróconej formie.

8.1 Σ_0 zamiast σ_0

Klasyczny jzyk AR posuguje si pojedynczym wyrónionym stanem pocztkowym σ_0 . Warunek Z7 (dopuszczalno opisów czciowych) wymusza przejcie do *zbioru* stanów pocztkowych Σ_0 : zdania **initially** mog nie ustala wartoci wszystkich fluentów, wic z dziedzin moe by zgodnych wiele stanów jednoczenie. Operacyjnie Σ_0 wyznaczamy z samych zda **initially** (formua w §4.2, pseudokod w §9.3); pozostaie zdania specyfikacyjne — **after** oraz **observable ... after** — traktujemy wycznie jako warunki poprawnoci modelu (M2, §6.4), a nie jako dodatkowy filtr stanów pocztkowych. Sprawdzenie tych warunków odbywa si ju na gotowym modelu, w trakcie ewaluacji kwerend (§9.7); jeeli dla pewnego $\sigma_0 \in \Sigma_0$ warunek M2 zawodzi, dziedzina po prostu nie ma modelu (§6.5). Przeniesienie tej konstrukcji na jzyk kwerend oznacza, e kwantyfikator „**necessary**” rozciga si na wszystkie $\sigma_0 \in \Sigma_0$ (patrz §7.3); scenariusz YSP (§10.1), w którym nieustalona warto *loaded* daje $|\Sigma_0| = 2$, ilustruje t konstrukcj w praktyce.

8.2 Mechanizm AC zamiast rozcznoci z trci projektu

Tre zadania definiuje wykonalno akcji zoonej przez warunek rozcznoci zbiorów zmiennych, na które wpywaj jej skadowe. Przyjlmymy ogólniejszy mechanizm zaczerpnięty z jzyka AC — detekcj konfliktów i generacj dekompozycji (§5) — z trzech powodów: (a) rozczo nie rozrónia akcji pewnych (**causes**) i niedeterministycznych (**releases**), przez co nie obsuguje poprawnie Przypadku 2 konfliktu; (b) nie uwzględnia ramifikacji wymuszanych zdaniami **always**, które mog rozszerzy faktyczny zbiór zmienianych fluentów; (c) sam warunek rozcznoci jest szczególnym przypadkiem mechanizmu AC (formalny argument w §5.6). Wybór ten zosta uzgodniony z prowadzc przedmiot.

8.3 $\text{Res}(\mathbb{A}, \sigma) = \emptyset$ przy braku wykonalnych skadowych

Jeeli w stanie σ adna akcja prosta $A_i \in \mathbb{A}$ nie jest indywidualnie wykonalna, to $V_{\text{exec}} = \emptyset$, a zbiór dekompozycji jest pusty (§9.5). Alternatyw byoby zinterpretowanie akcji zoonej o pustej dekompozycji jako akcji identycznociowej ($\{\sigma\}$), co czynioby $\mathbb{A}$ „wykonaln” nawet wtedy, gdy adna jej skadowa nie jest wykonalna. Przyjmujemy odwrotn konwencj: $\Delta(\mathbb{A}, \sigma) = \emptyset \Rightarrow \text{Res}(\mathbb{A}, \sigma) = \emptyset$. Decyzja ta jest spójna z traktowaniem akcji zoonej jako zbioru dziaa wymagajcych jednoczesnego wykonania i stanowi wewntrzne dopenienie reguy priorytetu **impossible** na poziomie akcji prostej (patrz §8.4).

8.4 Semantyka przypadków brzegowych

Dwa przypadki brzegowe w semantyce wymagaj jawnego rozstrzygniicia, poniewa formalna definicia pozostawia je niejednoznaczny.

Priorytet impossible nad efektem pustym. Jeeli w stanie σ aktywne jest zdanie **impossible** A if π , a jednoczenie adna regua **causes** dla A nie ma spenionego warunku, to wprost z definicji Res_0 otrzymalibymy $\text{Res}_0(A, \sigma) = \Sigma$, czyli akcja zachowywaaby si jak „nic nie robica”. Przyjmujemy, e **impossible** ma priorytet semantyczny: $\text{Res}(A, \sigma) = \emptyset$ niezalenie od regu efektowych. Odpowiada to klasycznej interpretacji **impossible** A if π jako skrót dla A **causes** $\perp$ if π (patrz §4.6, §9.2).

Kwerendy wartociowe a niewykonalno procesu. Jeeli proces $\mathbb{P}$ nie jest wykonalny z pewnego $\sigma_0 \in \Sigma_0$, to funkcja Ψ nie jest dla niego okrelona. Przyjmujemy, e **necessary** γ **after** $\mathbb{P}$ jest wtedy *faszywa*, poniewa nie mona stwierdzi, i γ zachodzi po wykonaniu procesu, który si nie koczy; symetrycznie **possibly** γ **after** $\mathbb{P}$ wymaga istnienia co najmniej jednej *penej* cieki koczcej si w stanie speniajcy γ . W szczególności **necessary** γ **after** $\mathbb{P}$ implikuje **necessary executable** $\mathbb{P}$. Odpowiadajca procedura obliczeniowa znajduje si w §9.7.

9 Algorytmizacja (pseudokod)

W tej sekcji pokazujemy, w jaki sposób definicje semantyczne z poprzednich rozdziaów mona przeooy na konkretne procedury obliczeniowe. Przyjmujemy podejcie jawne, oparte na penej eksploracji przestrzeni stanów oraz wszystkich istotnych kombinacji akcji. Nie jest to rozwizanie asymptotycznie optymalne, ale ma dwie wane zalety. Po pierwsze, bardzo dobrze odpowiada przytej semantyce formalnej. Po drugie, dla rozmiarów przykadów rozwaanych w projekcie pozostaje cakowicie wykonalne obliczeniowo.

W dalszej czci zakadamy, e formuy zdaniowe s reprezentowane w postaci drzew skadniowych, dziki czemu sprawdzenie, czy stan σ spenia formu α , moe by realizowane przez standardow rekurencyjn procedur ewaluacji. Zakadamy równie istnienie pomocniczej procedury przeksztaania formuy do postaci DNF, potrzebnej przy wykrywaniu konfliktów.

9.1 Generacja Σ

Pierwszym krokiem jest wyznaczenie zbioru stanów dopuszczalnych Σ . Zgodnie z definicj z §4 stan dopuszczalny to takie wartociowanie fluentów, które spenia wszystkie ograniczenia integralnoci postaci **always**. Poniewa kady fluent przyjmuje jedn z dwóch wartoci logicznych, dla zbioru fluentów $\mathcal{F}$ istnieje dokadnie

$$2^{|\mathcal{F}|}$$

moliwych wartociowa. Algorytm polega wic na przejsciu po wszystkich tych wartociowaniach i odrzuceniu tych, które naruszaj cho jedno ograniczenie integralnoci.

Formalnie:

$$\Sigma = \{\sigma : \forall(\text{always } \alpha) \in D, \sigma \models \alpha\}.$$

Po wygenerowaniu Σ w analogiczny sposób wyznaczamy zbiór stanów pocztkowych:

$$\Sigma_0 = \{\sigma \in \Sigma : \forall(\text{initially } \alpha) \in D, \sigma \models \alpha\}.$$

Warto podkreślić, że w klasie $\mathbb{DS4}$ zbiór Σ_0 może zawierać więcej niż jeden stan, ponieważ opis pocztkowy może być czciowy. W konsekwencji dalsze algorytmy muszą brać pod uwagę wszystkie stany pocztkowe zgodne z dziedziną.

Pseudokod dla algorytmu przedstawiono w algorytmie 1.

Algorithm 1 Pseudokod dla algorytmu generującego zbiór stanów dopuszczalnych

```

1: procedure GENERUJSIGMA( $\mathcal{F}, D$ )
2:    $\mathcal{V} \leftarrow$  wszystkie wartościowania  $\sigma: \mathcal{F} \rightarrow \{0, 1\}$ 
3:    $\Sigma \leftarrow \emptyset$ 
4:   for all  $\sigma \in \mathcal{V}$  do
5:      $ok \leftarrow \text{true}$ 
6:     for all zdanie  $(\text{always } \alpha) \in D$  do
7:       if  $\sigma \not\models \alpha$  then
8:          $ok \leftarrow \text{false}$ ; break
9:     if  $ok$  then
10:      dodaj  $\sigma$  do  $\Sigma$ 
return  $\Sigma$ 

```

Uwagi o złożoności. Jeżeli $m = |\mathcal{F}|$, to liczba wszystkich kandydatów na stan wynosi 2^m . Otrzymujemy zatem koszt rzędu

$$O(2^{|\mathcal{F}|} \cdot c_{\text{always}})$$

dla wyznaczenia Σ , gdzie c_{always} oznacza koszt sprawdzenia wszystkich zdań integralnociowych. Wyznaczenie Σ_0 ma koszt

$$O(|\Sigma| \cdot c_{\text{initially}}).$$

9.2 Res dla akcji prostej

Dla akcji prostej $A \in \mathcal{A}$ oraz stanu $\sigma \in \Sigma$ chcemy obliczyć zbiór wszystkich możliwych stanów wynikowych $\text{Res}(A, \sigma)$. Procedura ta jest bezpośrednią implementacją definicji z §4 i przebiega w trzech etapach: najpierw sprawdzana jest niewykonalność, następnie budowany jest zbiór kandydatów $\text{Res}_0(A, \sigma)$, a na końcu wykonywana jest minimalizacja względem funkcji New.

W pierwszym kroku sprawdzamy, czy akcja jest w stanie σ niewykonalna. Jeżeli istnieje reguła

$$(\text{impossible } A \text{ if } \pi) \in D$$

taka, że

$$\sigma \models \pi,$$

to zgodnie z przyjętą semantyką przyjmujemy od razu:

$$\text{Res}(A, \sigma) = \emptyset.$$

Jeżeli akcja nie jest niewykonalna, wyznaczamy zbiór wszystkich aktywnych efektów bezpośrednich oraz zbiór aktywnych zwolnień spod inercji:

$$E(A, \sigma) = \{\alpha : (A \text{ causes } \alpha \text{ if } \pi) \in D \text{ oraz } \sigma \models \pi\},$$

$$R(A, \sigma) = \{f : (A \text{ releases } f \text{ if } \pi) \in D \text{ oraz } \sigma \models \pi\}.$$

Następnie przeglądamy wszystkie stany dopuszczalne $\sigma' \in \Sigma$ i pozostawiamy tylko te, które spełniają wszystkie aktywne efekty z $E(A, \sigma)$. W ten sposób otrzymujemy zbiór kandydatów:

$$\text{Res}_0(A, \sigma) = \{\sigma' \in \Sigma : \forall \alpha \in E(A, \sigma), \sigma' \models \alpha\}.$$

Dla każdego kandydata $\sigma' \in \text{Res}_0(A, \sigma)$ obliczamy następnie zbiór

$$\text{New}(A, \sigma, \sigma'),$$

czyli zbiór literałów nowych względem stanu wejściowego. Zgodnie z definicją z §4 do zbioru tego trafiają:

1. literały odpowiadające fluentom inercyjnym, które rzeczywiście zmieniły wartość,
2. literały odpowiadające fluentom zwolnionym spod inercji przez aktywne reguły **releases**.

Na końcu z $\text{Res}_0(A, \sigma)$ zostawiamy tylko te stany, dla których zbiór New jest minimalny względem zawierania:

$$\text{Res}(A, \sigma) = \{\sigma' \in \text{Res}_0(A, \sigma) : \nexists \sigma'' \in \text{Res}_0(A, \sigma) (\text{New}(A, \sigma, \sigma'') \subset \text{New}(A, \sigma, \sigma'))\}.$$

Pseudokod dla algorytmu przedstawiono w algorytmie 2.

Algorithm 2 Pseudokod dla algorytmu wyznaczający funkcję Res

```
1: procedure GENERUJRES0( $A, \sigma, E$ )
2:    $\text{Res}_0 \leftarrow \emptyset$ 
3:   for all  $\sigma' \in \Sigma$  do
4:     if  $\sigma' \models \alpha$  dla każdego  $\alpha \in E$  then
5:       dodaj  $\sigma'$  do  $\text{Res}_0$ 
6:   return  $\text{Res}_0$ 
7: procedure GENERUJNEW( $A, \sigma, \sigma', R$ )
8:    $\text{New} \leftarrow \emptyset$ 
9:   for all  $f \in \mathcal{F}$  do
10:     $l \leftarrow$  ten spośród literałów  $\{f, \neg f\}$ , który spełnia  $\sigma' \models l$ 
11:    if  $f \in \mathcal{F}_I$  and  $\sigma(f) \neq \sigma'(f)$  then
12:      dodaj  $l$  do  $\text{New}$ 
13:    else if  $f \in R$  then
14:      dodaj  $l$  do  $\text{New}$ 
15:   return  $\text{New}$ 
16: procedure GENERUJRES( $A, \sigma$ )
17:   for all (impossible  $A$  if  $\pi$ )  $\in D$  do
18:     if  $\sigma \models \pi$  then
19:       return  $\emptyset$ 
20:    $E \leftarrow \emptyset, R \leftarrow \emptyset$ 
21:   for all ( $A$  causes  $\alpha$  if  $\pi$ )  $\in D$  do
22:     if  $\sigma \models \pi$  then
23:       dodaj  $\alpha$  do  $E$ 
24:   for all ( $A$  releases  $f$  if  $\pi$ )  $\in D$  do
25:     if  $\sigma \models \pi$  then
26:       dodaj  $f$  do  $R$ 
27:    $\text{Res}_0 \leftarrow \text{GENERUJRES0}(A, \sigma, E)$ 
28:   for all  $\sigma' \in \text{Res}_0$  do
29:      $\mathcal{N}[\sigma'] \leftarrow \text{GENERUJNEW}(A, \sigma, \sigma', R)$ 
30:    $\text{Res} \leftarrow \text{Res}_0$ 
31:   for all  $\sigma' \in \text{Res}_0$  do
32:     if istnieje  $\sigma'' \in \text{Res}_0$  takie, że  $\mathcal{N}[\sigma''] \subsetneq \mathcal{N}[\sigma']$  then
33:       usu  $\sigma'$  z  $\text{Res}$ 
34:   return  $\text{Res}$ 
```

Uwagi implementacyjne. Minimalizacja nie odbywa się względem liczności zbioru New, lecz względem relacji zawierania. Oznacza to, że dwa różne stany wynikowe mogą pozostać równocześnie dopuszczalne, jeżeli odpowiadające im zbiory zmian są nieporównywalne.

Uwagi o złożoności. Jeżeli oznaczymy przez $N = |\Sigma|$ liczbę stanów dopuszczalnych, to koszt wyznaczenia $\text{Res}_0(A, \sigma)$ jest liniowy względem N , natomiast końcowa minimalizacja wymaga w najprostszym wariantcie porównania kandydatów parami. W rezultacie naiwny koszt obliczenia $\text{Res}(A, \sigma)$ jest rzędu

$$O(N \cdot c_{\text{efekty}} + |\text{Res}_0(A, \sigma)|^2 \cdot |\mathcal{F}|).$$

9.3 Generacja Σ_0

Zgodnie z przyjętą konwencją (§4.2, §8.1) zbiór Σ_0 jest wyznaczony wyłącznie przez zdania typu **initially**. Zdania wartościowe (**after**) oraz obserwacyjne (**observable ... after**) nie pełni roli filtru

Σ_0 , lecz s warunkami poprawności modelu (M2); ich sprawdzenie następuje na etapie ewaluacji kwerend (§9.7) w oparciu o ju zbudowany model.

Algorytm polega zatem na kolejnym przesiewaniu zbioru Σ przez poszczególne zdania **initially**: dla kadego takiego zdania zatrzymujemy wycznie te stany, które speniaj jego formu. Pseudokod przedstawiono w algorytmie 3.

Algorithm 3 Pseudokod dla algorytmu wyznaczającego stany pocztkowe

```

1: procedure GENERUJSIGMA0( $\Sigma, D$ )
2:    $\Sigma_0 \leftarrow \Sigma$ 
3:   for all (initially  $\alpha$ )  $\in D$  do
4:      $\Sigma_0 \leftarrow \{\sigma \in \Sigma_0 : \sigma \models \alpha\}$ 
5:   return  $\Sigma_0$ 

```

Uwagi o zoonoci. Koszt procedury wynosi $O(|\Sigma| \cdot c_{\text{initially}})$, gdzie $c_{\text{initially}}$ oznacza czny koszt sprawdzenia wszystkich zda **initially** w pojedynczym stanie.

9.4 Wykrywanie konfliktów

Dla akcji zoonej

$$\mathbb{A} = \{A_1, \dots, A_k\}$$

konieczne jest wykrycie konfliktów pomiędzy parami akcji prostych. W tym miejscu korzystamy z idei wprowadzonych dla języka AC, ale w wersji dostosowanej do klasy $\mathbb{DS4}$. Poniewa w naszej dziedzinie wystpuj wycznie akcje proste, konflikty badamy między parami akcji prostych należych do $\mathbb{A}$.

Zgodnie z definicj z §5 konflikt zawsze badamy względem konkretnego stanu σ , poniewa to od niego zależy, które reguły **causes** oraz **releases** s aktywne.

Dla kadej pary rónych akcji $A_i, A_j \in \mathbb{A}$ wykonujemy następujące kroki:

1. wyznaczamy aktywne efekty obu akcji w stanie σ ,
2. sprowadzamy te efekty do postaci DNF,
3. sprawdzamy, czy zachodzi który z dwóch przypadków konfliktu:
 - (a) **causes–causes**: istnieją skadniki DNF zawierające literay komplementarne,
 - (b) **causes–releases**: fluent zwolniony przez jedn akcję występuje w skadniku DNF efektu drugiej akcji.

Poniewa relacja konfliktu jest symetryczna, wynik tej procedury można reprezentować przez nieskierowany graf konfliktu

$$G_\sigma(\mathbb{A}) = (V, E),$$

gdzie wierzchołkami s akcje proste należące do $\mathbb{A}$, a krawędź między A_i i A_j oznacza, że akcje te s w konflikcie w stanie σ .

Pseudokod dla algorytmu przedstawiono w algorytmie 4.

Algorithm 4 Pseudokod dla algorytmu wykrywającego konflikty

```
1: procedure WYKRYJKONFLIKTY( $\mathbb{A}, \sigma, D$ )
2:    $E_G \leftarrow \emptyset$ 
3:   for all para różnych akcji  $A_i, A_j \in \mathbb{A}$  do
4:     wyznacz aktywne efekty i aktywne zwolnienia obu akcji w stanie  $\sigma$ 
5:     zamie aktywne efekty obu akcji na DNF
6:     if zachodzi konflikt typu causes-causes or konflikt typu causes-releases then
7:       dodaj krawd  $\{A_i, A_j\}$  do  $E_G$ 
   return  $G_\sigma(\mathbb{A}) = (\mathbb{A}, E_G)$ 
```

Uwagi o zoonoci. Dla $k = |\mathbb{A}|$ liczba par akcji jest rzędu $O(k^2)$. Główny koszt pojedynczego porównania zależy od zoonoci konwersji formu do DNF oraz od liczby skadników w tej postaci. W naszych przykładach formuły efektów są krótkie, dlatego koszt ten pozostaje niewielki.

9.5 Generacja dekompozycji

Po zbudowaniu grafu konfliktu wyznaczamy dekompozycje akcji zoonej. Korzystamy tutaj z definicji przyjętej w §5 dla klasy DS4. Oznacza to, że dekompozycja $\delta \in \Delta(\mathbb{A}, \sigma)$ jest maksymalnym podzbiorem akcji prostych należących do $\mathbb{A}$, który:

1. składa się wyłącznie z akcji indywidualnie wykonalnych w stanie σ ,
2. nie zawiera par akcji pozostających ze sobą w konflikcie,
3. jest maksymalny względem inkluzji.

Najprostsza implementacja polega na enumeracji wszystkich podzbiorów akcji. Najpierw odrzucamy akcje indywidualnie niewykonalne, a następnie spośród pozostałych podzbiorów wybieramy te, które są bezkonfliktowe i maksymalne.

Niech

$$V_{\text{exec}} = \{A \in \mathbb{A} : \text{Res}(A, \sigma) \neq \emptyset\}$$

oznacza zbiór akcji prostych wykonalnych indywidualnie w stanie σ . Na zbiorze tym budujemy graf konfliktu $G_\sigma(\mathbb{A})$. Dekompozycje są wtedy dokładnie maksymalnymi bezkonfliktowymi podzbiórmi zbioru V_{exec} .

Pseudokod dla algorytmu przedstawiono w algorytmie 5.

Algorithm 5 Pseudokod dla algorytmu generującego dekompozycje

```
1: procedure GENERUJDEKOMPOZYCJE( $\mathbb{A}, \sigma, D$ )
2:    $V_{\text{exec}} \leftarrow \emptyset$ 
3:   for all  $A \in \mathbb{A}$  do
4:     if  $\text{Res}(A, \sigma) \neq \emptyset$  then
5:       dodaj  $A$  do  $V_{\text{exec}}$ 
6:   if  $V_{\text{exec}} = \emptyset$  then return  $\emptyset$ 
7:   zbuduj graf konfliktu  $G_\sigma(\mathbb{A})$  na zbiorze  $V_{\text{exec}}$ 
8:    $\mathcal{C} \leftarrow \emptyset$ 
9:   for all podzbiór  $X \subseteq V_{\text{exec}}$  do
10:    if adna para akcji z  $X$  nie jest połączona krawdzą konfliktu then
11:      dodaj  $X$  do  $\mathcal{C}$ 
12:   usu z  $\mathcal{C}$  wszystkie zbiory, które nie są maksymalne względem inkluzji
   return  $\Delta(\mathbb{A}, \sigma) = \mathcal{C}$ 
```

Jeżeli zbiór V_{exec} jest pusty, to nie istnieje adna dekompozycja, a więc przyjmujemy

$$\Delta(\mathbb{A}, \sigma) = \emptyset \quad \text{oraz} \quad \text{Res}(\mathbb{A}, \sigma) = \emptyset.$$

Uwagi o zoonoci. Dla $k = |\mathbb{A}|$ liczba wszystkich podzbiorów wynosi 2^k . Sprawdzenie bezkonfliktowości pojedynczego podzbioru można wykonać w czasie $O(k^2)$, stąd prosty koszt rzędu

$$O(2^{|\mathbb{A}|} \cdot |\mathbb{A}|^2).$$

Jest to koszt wykładniczy, ale dla niewielkich przykładów projektowych pozostaje akceptowalny.

9.6 Res dla akcji zoonej

Po wyznaczeniu zbioru dekompozycji $\Delta(\mathbb{A}, \sigma)$ możemy obliczyć wynik akcji zoonej. W tym miejscu również stosujemy adaptację idei z języka AC do klasy $\mathbb{DS4}$. Ponieważ w opisie dziedziny występują wyłącznie akcje proste, efekty danej dekompozycji otrzymujemy przez poczenie aktywnych efektów i aktywnych zwolnień wszystkich jej składowych.

Dla każdej dekompozycji

$$\delta = \{A_{i_1}, \dots, A_{i_r}\} \in \Delta(\mathbb{A}, \sigma)$$

definiujemy:

$$E(\delta, \sigma) = \bigcup_{A \in \delta} E(A, \sigma), \quad R(\delta, \sigma) = \bigcup_{A \in \delta} R(A, \sigma).$$

Następnie:

$$\text{Res}_0(\delta, \sigma) = \{\sigma' \in \Sigma : \forall \alpha \in E(\delta, \sigma), \sigma' \models \alpha\}.$$

Dla każdego $\sigma' \in \text{Res}_0(\delta, \sigma)$ obliczamy zbiór

$$\text{New}(\delta, \sigma, \sigma')$$

analogicznie jak dla akcji prostej, z tą różnicą, że uwzględniamy również wszystkie zwolnienia wynikające z akcji należących do δ . Następnie zachowujemy wyłącznie stany minimalne:

$$\text{Res}(\delta, \sigma) = \{\sigma' \in \text{Res}_0(\delta, \sigma) : \nexists \sigma'' \in \text{Res}_0(\delta, \sigma) (\text{New}(\delta, \sigma, \sigma'') \subset \text{New}(\delta, \sigma, \sigma'))\}.$$

Ostatecznie wynik akcji zoonej jest sumą wyników wszystkich jej dekompozycji:

$$\text{Res}(\mathbb{A}, \sigma) = \bigcup_{\delta \in \Delta(\mathbb{A}, \sigma)} \text{Res}(\delta, \sigma).$$

Pseudokod dla algorytmu przedstawiono w algorytmie 6.

Algorithm 6 Pseudokod dla algorytmu generującego funkcję Res dla akcji zoonej

```

1: procedure GENERUJRESZLOZONE( $\mathbb{A}, \sigma, D$ )
2:    $\Delta(\mathbb{A}, \sigma) \leftarrow \text{GenerujDekompozycje}(\mathbb{A}, \sigma, D)$ 
3:   if  $\Delta(\mathbb{A}, \sigma) = \emptyset$  then return  $\emptyset$ 
4:    $R_{\text{all}} \leftarrow \emptyset$ 
5:   for all  $\delta \in \Delta(\mathbb{A}, \sigma)$  do
6:     wyznacz  $E(\delta, \sigma)$  oraz  $R(\delta, \sigma)$ 
7:     zbuduj  $\text{Res}_0(\delta, \sigma)$ 
8:     for all  $\sigma' \in \text{Res}_0(\delta, \sigma)$  do
9:       oblicz  $\text{New}(\delta, \sigma, \sigma')$ 
10:    wyznacz  $\text{Res}(\delta, \sigma)$  przez minimalizację względem zawierania
11:     $R_{\text{all}} \leftarrow R_{\text{all}} \cup \text{Res}(\delta, \sigma)$ 
return  $R_{\text{all}}$ 

```

9.7 Ewaluacja kwerend

Ewaluacj kwerend najwygodniej oprze na drzewie wykonania procesu. Korze takiego drzewa odpowiada wszystkim stanom pocztkowym z Σ_0 . Na poziomie i przechowujemy wszystkie stany osignite po wykonaniu pierwszych i kroków procesu, a midzy kolejnymi poziomami zapisujemy relacj przejcia wynikajc z funkcji Res.

Taka reprezentacja jest szczególnie istotna w przypadku kwerend typu **necessary**. Sam zbiór stanów osigalnych po kolejnych krokach nie wystarcza, poniewa moe si zdarzy, e cz cieek zakoczy si wczesniej brakiem nastpnika, mimo e inne cieki nadal prowadz do stanu kocowego. Dlatego przy ewaluacji kwerend koniecznych naley bada cae drzewo wykonania, a nie tylko kocowy zbiór stanów.

Poniewa w projektowanym jzyku kwerend rozwaamy procesy rozpoczynane ze zbioru stanów pocztkowych Σ_0 , przyjmujemy nastpujc interpretacj czterech podstawowych typów kwerend:

1. **possibly executable** $\mathbb{P}$,
2. **necessary executable** $\mathbb{P}$,
3. **possibly γ after** $\mathbb{P}$,
4. **necessary γ after** $\mathbb{P}$.

Kwerenda **possibly executable** jest prawdziwa wtedy i tylko wtedy, gdy istnieje co najmniej jedna pena cieka wykonania procesu od pewnego stanu pocztkowego do poziomu n .

Kwerenda **necessary executable** jest prawdziwa wtedy i tylko wtedy, gdy kada cieka rozpoczta w dowolnym stanie pocztkowym moe zosta doprowadzona do koca procesu. Algorytmicznie oznacza to, e w drzewie wykonania nie istnieje wze na poziomie mniejszym ni n , który nie miaby adnego nastpnika.

Kwerenda **possibly γ after** $\mathbb{P}$ jest prawdziwa wtedy i tylko wtedy, gdy istnieje przynajmniej jedna pena cieka wykonania, której stan kocowy spenia formu γ .

Kwerenda **necessary γ after** $\mathbb{P}$ jest prawdziwa wtedy i tylko wtedy, gdy proces jest koniecznie wykonalny oraz kady stan kocowy kadej peney cieki spenia formu γ . Przyjmujemy wic, e niewykonalno procesu na cho jednej ciece powoduje faszywo tej kwerendy.

Pseudokod dla algorytmu przedstawiono w algorytmie 7.

Algorithm 7 Pseudokod dla algorytmu wykonujcego kwerend

- 1: **procedure** KWERENDA(Q, D)
 - 2: zbuduj drzewo wykonania procesu od wszystkich stanów Σ_0
 - 3: **if** pytanie ma posta **possibly executable** $\mathbb{P}$ **then return true** wtedy i tylko wtedy, gdy istnieje pena cieka dugoci n
 - 4: **if** pytanie ma posta **necessary executable** $\mathbb{P}$ **then return true** wtedy i tylko wtedy, gdy kada cieka dochodzi do poziomu n
 - 5: **if** pytanie ma posta **possibly γ after** $\mathbb{P}$ **then return true** wtedy i tylko wtedy, gdy istnieje pena cieka koczca si w stanie speniajcy γ
 - 6: **if** pytanie ma posta **necessary γ after** $\mathbb{P}$ **then return true** wtedy i tylko wtedy, gdy kada pena cieka koczy si w stanie speniajcy γ oraz nie istnieje cieka zablokowana przed kocem procesu
-

9.8 Uwagi o zoonoci

Niech:

- $m = |\mathcal{F}|$ oznacza liczb fluentów,

- $N = |\Sigma| \leq 2^m$ liczb stanów dopuszczalnych,
- $k = |\mathbb{A}|$ liczb akcji prostych w pojedynczym kroku akcji zoonej,
- n dugo procesu.

Wtedy:

- generacja Σ jest wykładnicza względem m ,
- obliczenie $\text{Res}(A, \sigma)$ dla akcji prostej jest wielomianowe względem N ,
- generacja dekompozycji akcji zoonej jest wykładnicza względem k ,
- ewaluacja całego procesu może być wykładnicza względem długości procesu, ponieważ liczba cieków niedeterministycznych może szybko rosnąć.

Z praktycznego punktu widzenia nie stanowi to jednak istotnego ograniczenia dla projektu. Rozważane przez nas przykłady zawierają niewielką liczbę fluentów i akcji, dlatego pełna eksploracja przestrzeni stanów, dekompozycji oraz cieków wykonania pozostaje w pełni akceptowalna. Jednocześnie taka implementacja jest przejrzysta, atawa do uzasadnienia formalnie i bezporównanie odpowiada definicjom semantycznym wprowadzonym wcześniej.

10 Przykłady

10.1 YSP z rosyjską ruletką

Scenariusz oparty na rosyjskim indyku, ilustrujący niedeterminizm (**releases**), niewykonalność (**impossible**) i czciowy opis stanu poczkowego.

Sygnatura:

$$\begin{aligned}\mathcal{F} &= \{loaded, alive\}, \\ \mathcal{A} &= \{Load, Shoot, Spin\}.\end{aligned}$$

Dziedzina D :

initially <i>alive</i>	(stan broni nieznany — opis czciowy)
<i>Load</i> causes <i>loaded</i>	
impossible <i>Load</i> if <i>loaded</i>	(nie można naładować naładowanej)
<i>Shoot</i> causes $\neg loaded$	
<i>Shoot</i> causes $\neg alive$ if <i>loaded</i>	
<i>Spin</i> releases <i>loaded</i>	(kręcenie bbenkiem — niedeterminizm)
$\neg loaded$ after <i>Shoot</i>	(specyfikacja: po strzale broń rozładowana)

Stany dopuszczalne (brak **always**, wszystkie 4 stany dopuszczalne):

$$\sigma_0 = \{loaded, alive\}, \quad \sigma_1 = \{loaded, \neg alive\}, \quad \sigma_2 = \{\neg loaded, alive\}, \quad \sigma_3 = \{\neg loaded, \neg alive\}$$

Stany poczkowe. Zdanie **initially** *alive* wybiera z Σ dwa stany: $\Sigma_0 = \{\sigma_0, \sigma_2\}$ (*loaded* nieokreślone — opis czciowy, Z7).

Sprawdzenie M2. Zdanie wartociowe $\neg loaded$ **after** *Shoot* nie filtruje Σ_0 (zgodnie z decyzją z §8.1), lecz wymaga weryfikacji na zbudowanym modelu. Dla σ_0 : $\text{Res}(Shoot, \sigma_0) = \{\sigma_3\} \models \neg loaded$ (akcja rozładowuje i zabija); dla σ_2 : $\text{Res}(Shoot, \sigma_2) = \{\sigma_2\} \models \neg loaded$ (efekt pusty — broń już rozładowana). Oba łady spełniają zdanie, więc M2 zachodzi i dziedzina ma model.

Proces $P = (\{Load\}, \{Spin\}, \{Shoot\})$.

Z σ_0 : *Load* jest *niewykonalne* (bo $\sigma_0 \models loaded$, a **impossible** *Load if loaded*). Proces nie moe startowa z σ_0 .

Z σ_2 :

1. *Load* w σ_2 : efekt *loaded* $\Rightarrow \sigma_0 = \{loaded, alive\}$.
2. *Spin* w σ_0 : fluent *loaded* jest zwolniony (**releases**). Oba stany σ_0 i σ_2 zmieniaj si minimalnie, wic $Res(Spin, \sigma_0) = \{\sigma_0, \sigma_2\}$ (niedeterminizm).
3. *Shoot* po kadej gazi:
 - ze σ_0 (bro naadowana): efekty $\neg loaded \wedge \neg alive \Rightarrow \sigma_3$.
 - ze σ_2 (bro pusta): efekt $\neg loaded$ ju speniony, warunek **if loaded** faszywy $\Rightarrow$ efekt pusty, σ_2 bez zmian.

Stany kocowe: $\{\sigma_2, \sigma_3\}$.

Kwerendy:

- **necessary executable** P — z $\sigma_0 \in \Sigma_0$ pierwszy krok jest niewykonalny. **NIE**.
- **possibly executable** P — z σ_2 proces przechodzi wszystkie kroki. **TAK**.
- **necessary $\neg alive$ after** P — ga przez σ_2 (po *Spin*) koczy si w σ_2 z *alive*. **NIE**.
- **possibly $\neg alive$ after** P — ga przez σ_0 (po *Spin*) koczy si w σ_3 . **TAK**.

10.2 Producent i konsumenci

Scenariusz oparty na przykadzie 5.8. W odrónieniu od oryginau nie blokujemy sytuacji, w której obaj konsumenci pobieraj produkt jednoczenie — w $\mathbb{DS4}$ nie mona wyrazi **impossible** dla akcji zoonej. Daje to dodatkow dekompozycj i dodatkowy niedeterminizm.

Producent napenia bufor, dwaj konsumenci A i B pobieraj z niego produkt i zuywaj go.

Sygnatura:

$$\begin{aligned}\mathcal{F} &= \{empty, hasA, hasB\}, \\ \mathcal{A} &= \{Put, GetA, GetB, ConsumeA, ConsumeB\}.\end{aligned}$$

Dziedzina D :

initially $\neg empty \wedge hasA \wedge hasB$	(bufor peny, obaj konsumenci z produktem)
Put causes $\neg empty$	(producent napenia bufor)
GetA causes $empty \wedge hasA$ if $\neg empty$	(A pobiera, o ile jest co pobra)
GetB causes $empty \wedge hasB$ if $\neg empty$	(B pobiera, o ile jest co pobra)
ConsumeA causes $\neg hasA$ if $hasA$	(A zuywa swój produkt)
ConsumeB causes $\neg hasB$ if $hasB$	(B zuywa swój produkt)

Brak **always** — wszystkie $2^3 = 8$ stanów jest dopuszczalnych. Stan pocztkowy: $\Sigma_0 = \{\sigma\}$ gdzie $\sigma = \{\neg empty, hasA, hasB\}$ (bufor peny, obaj konsumenci maj produkt).

Akcja zoona $\mathbb{A} = \{Put, GetA, GetB, ConsumeA, ConsumeB\}$ w σ .

Konflikty w σ (oba warunki wstpne spenione):

- *Put* vs. *GetA*: *Put* daje $\neg empty$, *GetA* daje $empty \wedge hasA$ — literay $\neg empty$ i $empty$ s komplementarne. **Konflikt** (Przypadek 1).

- *Put* vs. *GetB*: analogicznie. **Konflikt**.
- *GetA* vs. *ConsumeA*: *GetA* daje $empty \wedge hasA$, *ConsumeA* daje $\neg hasA$ — $hasA$ i $\neg hasA$ komplementarne. **Konflikt**.
- *GetB* vs. *ConsumeB*: analogicznie. **Konflikt**.
- *GetA* vs. *GetB*: oba daj $empty$ (ten sam znak) i róne fluenty $hasA$, $hasB$ — **brak konfliktu**. W oryginalnym przykładzie 5.8 *GetA* i *GetB* s wykluczone zdaniem **impossible** $\{GetA, GetB\}$. W $\mathbb{DS4}$ **impossible** dotyczy tylko akcji prostych, wic tej pary nie moemy tak zablokować.
- *ConsumeA* vs. *ConsumeB*, *Put* vs. *ConsumeA/ConsumeB*, *GetA* vs. *ConsumeB*, *GetB* vs. *ConsumeA*: brak konfliktu.

Dekompozycje — maksymalne podzbiory bez konfliktu:

- $\Delta_1 = \{Put, ConsumeA, ConsumeB\}$ — nie da si dooy *GetA* ani *GetB* (oba koliduj z *Put*).
- $\Delta_2 = \{GetA, GetB\}$ — nie da si dooy *Put*, *ConsumeA* ani *ConsumeB* (konflikty). *Ta dekompozycja nie istnieje w oryginalnym przykładzie 5.8.*
- $\Delta_3 = \{GetA, ConsumeB\}$ — nie da si dooy *Put*, *GetB* ani *ConsumeA* (konflikty).
- $\Delta_4 = \{GetB, ConsumeA\}$ — analogicznie.

Wyniki z $\sigma = \{\neg empty, hasA, hasB\}$:

- $\Delta_1: \neg empty \wedge \neg hasA \wedge \neg hasB \Rightarrow \sigma_1$.
- $\Delta_2: empty \wedge hasA \wedge hasB \Rightarrow \sigma_2$.
- $\Delta_3: empty \wedge hasA \wedge \neg hasB \Rightarrow \sigma_3$.
- $\Delta_4: empty \wedge \neg hasA \wedge hasB \Rightarrow \sigma_4$.

$Res(\mathbb{A}, \sigma) = \{\sigma_1, \sigma_2, \sigma_3, \sigma_4\}$. Oryginalne AC daje tylko $\{\sigma_1, \sigma_3, \sigma_4\}$ — brak σ_2 , bo tam *GetA* i *GetB* s razem niemoliwe.

Proces $P = (\mathbb{A}, \{ConsumeA, ConsumeB\})$.

Krok 2 — $\{ConsumeA, ConsumeB\}$ w kadym stanie wynikowym (brak konfliktu, bo $\neg hasA$ i $\neg hasB$ dotycz rónych fluentów):

- $\sigma_1 = \{\neg e, \neg A, \neg B\}$: oba warunki ($hasA$, $hasB$) niespenione — efekt pusty, stan bez zmian.
- $\sigma_2 = \{e, A, B\}$: obaj konsumenci konsumuj $\Rightarrow \sigma_0 = \{e, \neg A, \neg B\}$.
- $\sigma_3 = \{e, A, \neg B\}$: tylko A konsumuje $\Rightarrow \sigma_0$.
- $\sigma_4 = \{e, \neg A, B\}$: tylko B konsumuje $\Rightarrow \sigma_0$.

(Uywamy skrótów $e = empty$, $A = hasA$, $B = hasB$.)

Po procesie P : $Res = \{\sigma_0, \sigma_1\}$.

Kwerendy:

- **necessary executable** P — oba kroki maj $Res \neq \emptyset$. **TAK**.
- **possibly empty after** P — σ_0 spenia $empty$. **TAK**.
- **necessary empty after** P — σ_1 spenia $\neg empty$. **NIE**.
- **necessary $\neg hasA \wedge \neg hasB$ after** P — zarówno σ_0 , jak i σ_1 speniaj $\neg hasA \wedge \neg hasB$. **TAK** — niezalenie od przebiegu, po procesie aden konsument nie ma produktu.

10.3 Dwa przeczniki, arówka i alarm

Nowy przykład rozszerzający dwuprzecznikowy scenariusz. Dwa przeczniki sw_1, sw_2 sterują arówką (wiatro wieci, gdy wczony jest przynajmniej jeden z nich), a gdy wieci wiatro, uruchamia się alarm (czujnik ruchu). Dodatkowo akcja *On* wymusza zapalenie wiatra (np. ręczne zwarcie obwodu), a akcja *Flicker* modeluje zakócenie zasilania — nie wymusza konkretnej wartości *light*, lecz zwalnia ten fluent spod inercji.

Scenariusz demonstruje: **always** (ograniczenia integralności i ramifikacje), **noninertial** (alarm zmienia się „za darmo” jako efekt uboczny), akcja *zoon* bez konfliktu oraz konflikt Przypadek 2 (**causes** vs. **releases**).

Sygnatura:

$$\begin{aligned}\mathcal{F} &= \{sw_1, sw_2, light, alarm\}, \\ \mathcal{A} &= \{Toggle_1, Toggle_2, On, Flicker\}.\end{aligned}$$

Dziedzina D :

always $light \leftrightarrow (sw_1 \vee sw_2)$	(ramifikacja: wiatro wieci, gdy wczony choć jeden przecznik)
always $light \rightarrow alarm$	(ramifikacja: wiecra arówka aktywuje alarm)
noninertial $alarm$	(alarm nie podlega inercji — reaguje na wiatro)
$Toggle_1$ causes sw_1 if $\neg sw_1$	(przecznik pierwszy: warunkowe causes)
$Toggle_1$ causes $\neg sw_1$ if sw_1	
$Toggle_2$ causes sw_2 if $\neg sw_2$	(przecznik drugi: warunkowe causes)
$Toggle_2$ causes $\neg sw_2$ if sw_2	
On causes $light$	(wymuszenie zapalenia wiatra)
$Flicker$ releases $light$	(zakócenie — wiatro może się zmienić)
initially $\neg sw_1 \wedge \neg sw_2$	(stan $alarm$ nieokreślony — opis czciowy)

Stany dopuszczalne Σ . Z $2^{|\mathcal{F}|} = 16$ wartościowa odrzucamy te, któreami zdania **always**. Po pierwsze, musi zachodzić $light \leftrightarrow (sw_1 \vee sw_2)$, co wyklucza 8 stanów i wyznacza wartość *light* na podstawie przeczników. Po drugie, gdy *light* jest prawdziwe, musi być prawdziwy również *alarm*, co eliminuje jeszcze 3 stany. Pozostaje 5 stanów dopuszczalnych:

$$\begin{aligned}\sigma_0 &= \{\neg sw_1, \neg sw_2, \neg light, \neg alarm\}, \\ \sigma_1 &= \{\neg sw_1, \neg sw_2, \neg light, alarm\}, \\ \sigma_2 &= \{\neg sw_1, sw_2, light, alarm\}, \\ \sigma_3 &= \{sw_1, \neg sw_2, light, alarm\}, \\ \sigma_4 &= \{sw_1, sw_2, light, alarm\}.\end{aligned}$$

Zdanie **initially** wymusza $\neg sw_1, \neg sw_2$, natomiast nie ustala wartości fluentu *alarm*. Z dopuszczalnych stanów warunek ten spełniają zatem dwa: σ_0 oraz σ_1 . Ostatecznie $\Sigma_0 = \{\sigma_0, \sigma_1\}$ — jest to przypadek opisu czciowego (Z7).

Proces $P = (\{Toggle_1, Toggle_2\}, \{On, Flicker\})$.

Krok 1: $\{Toggle_1, Toggle_2\}$ w σ_0 .

W stanie σ_0 aktywne są reguły $Toggle_i$ **causes** sw_i **if** $\neg sw_i$ (obu przeczników). Obie akcje są indywidualnie wykonalne.

Konflikty: efekty aktywne to sw_1 (DNF: $\{\{sw_1\}\}$) i sw_2 (DNF: $\{\{sw_2\}\}$). Literały dotyczą różnych fluentów, więc nie ma pary komplementarnej — brak konfliktu Przypadek 1. adna z akcji nie jest **releases**, więc brak Przypadek 2.

Dekompozycja: jedna, $\delta = \{Toggle_1, Toggle_2\}$.

Wynik: $E(\delta, \sigma_0) = \{sw_1, sw_2\}$, $R(\delta, \sigma_0) = \emptyset$.

$$\text{Res}_0(\delta, \sigma_0) = \{\sigma \in \Sigma : \sigma \models sw_1 \wedge sw_2\} = \{\sigma_4\}.$$

Jedyny kandydat, wic $\text{Res}(\{Toggle_1, Toggle_2\}, \sigma_0) = \{\sigma_4\}$ z $\text{New}(\sigma_0, \sigma_4) = \{sw_1, sw_2, light\}$. Zmiana *light* jest wymuszona przez **always** (ramifikacja), zmiana *alarm* jest „za darmo” — fluent nieinercyjny nie trafia do New.

Analogicznie dla drugiego stanu pocztkowego: w $\sigma_1 = \{\neg sw_1, \neg sw_2, \neg light, alarm\}$ aktywne reguły i zbiór $E(\delta, \sigma_1) = \{sw_1, sw_2\}$ s identyczne, wic $\text{Res}_0(\delta, \sigma_1) = \{\sigma_4\}$ i $\text{Res}(\{Toggle_1, Toggle_2\}, \sigma_1) = \{\sigma_4\}$. Obie gązie procesu zbiegaj si zatem w σ_4 ju po pierwszym kroku — nieokrelono stanu pocztkowego alarmu zostaa „wchonita” przez ramifikacj $light \rightarrow alarm$ w nowym stanie.

Krok 2: $\{On, Flicker\}$ w σ_4 .

Indywidualna wykonalno:

- $\text{Res}(\{On\}, \sigma_4)$: $E = \{light\}$, $R = \emptyset$. $\text{Res}_0 = \{\sigma_2, \sigma_3, \sigma_4\}$ (wszystkie stany z *light*). Obliczenia New:

$$\text{New}(\sigma_4, \sigma_2) = \{\neg sw_1\}, \quad \text{New}(\sigma_4, \sigma_3) = \{\neg sw_2\}, \quad \text{New}(\sigma_4, \sigma_4) = \emptyset.$$

Minimum to $\emptyset$, zatem $\text{Res}(\{On\}, \sigma_4) = \{\sigma_4\}$ (akcja bez efektu — wiatu ju wieci).

- $\text{Res}(\{Flicker\}, \sigma_4)$: $E = \emptyset$, $R = \{light\}$. $\text{Res}_0 = \Sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4\}$. Fluent *light* jest zwolniony — jego litera (zgodny z σ') wchodzi do New:

$$\begin{aligned} \text{New}(\sigma_4, \sigma_0) &= \{\neg sw_1, \neg sw_2, \neg light\}, \\ \text{New}(\sigma_4, \sigma_1) &= \{\neg sw_1, \neg sw_2, \neg light\}, \\ \text{New}(\sigma_4, \sigma_2) &= \{\neg sw_1, light\}, \\ \text{New}(\sigma_4, \sigma_3) &= \{\neg sw_2, light\}, \\ \text{New}(\sigma_4, \sigma_4) &= \{light\}. \end{aligned}$$

Minimum wzgldem inkluzji to $\{light\}$ (dla σ_4) oraz $\{\neg sw_1, \neg sw_2, \neg light\}$ (dla σ_0 i σ_1 , nieporównywalne z $\{light\}$). Zatem $\text{Res}(\{Flicker\}, \sigma_4) = \{\sigma_0, \sigma_1, \sigma_4\}$.

Konflikty: $E(On, \sigma_4) = \{light\}$, $\text{DNF}(light) = \{\{light\}\}$; *Flicker* zwalnia fluent *light*, który wystpuje w tym skadniku — to **konflikt Przypadek 2 (causes vs. releases)**.

Dekompozycja: graf konfliktu na $V_{\text{exec}} = \{On, Flicker\}$ ma jedn krawd $\{On, Flicker\}$. Maksymalne zbiory bezkonfliktowe to dwa singletony: $\delta_1 = \{On\}$, $\delta_2 = \{Flicker\}$.

Wynik:

$$\text{Res}(\{On, Flicker\}, \sigma_4) = \text{Res}(\delta_1, \sigma_4) \cup \text{Res}(\delta_2, \sigma_4) = \{\sigma_4\} \cup \{\sigma_0, \sigma_1, \sigma_4\} = \{\sigma_0, \sigma_1, \sigma_4\}.$$

Stany osigalne po procesie P : $\{\sigma_0, \sigma_1, \sigma_4\}$.

Kwerendy:

- **necessary executable P** — Z obu stanów pocztkowych σ_0, σ_1 krok 1 prowadzi do σ_4 , a krok 2 ma w σ_4 niepuste Res, wic kada cieka dochodzi do poziomu 2. **TAK**.
- **possibly executable P** — Natychmiast z powyszego. **TAK**.
- **necessary alarm after P** — σ_0 spenia $\neg alarm$, wic istnieje cieka koczca si bez alarmu. **NIE**.

- possibly *alarm* after P — σ_4 i σ_1 speniają *alarm*. **TAK**.

Komentarz do ramifikacji i nieinercji. W kroku 1 jedyna zmiana wymuszona bezporednio przez **causes** to sw_1, sw_2 ; zmiana *light* wynika z **always** i zostaje wliczona do New, natomiast *alarm* take si zmienia „w tle”, ale dziki **noninertial** nie obcia zbioru New. W kroku 2 *Flicker* pokazuje, e **releases** moe dawa wiecej ni jeden minimalny stan wynikowy nawet pojedynczo, a w poczeniu z *On* generuje konflikt Przypadek 2, co rozbija akcj zoon na dwie singletonowe dekompozycje.

10.4 Dwóch robotników wynoszących stół

Autorski scenariusz integrujący, obejmujący wszystkie mechanizmy dziedziny DS4: opis czciowy (Z7), ograniczenia integralności, fluenty nieinercyjne, ramifikacje, warunkowe **causes** i **releases**, **impossible** aktywowan dynamicznie, obie kategorie konfliktu akcji zoonej (Przypadek 1 i Przypadek 2) oraz zdanie obserwacyjne jako warunek poprawności modelu (M2). Na jednej dziedzinie porównamy dwa procesy P_1 i P_2 , tak by w sumie trafi wszystkie interesujące ciek.

Dwóch robotników, A i B , ma wynie stół z pokoju. Kady moe samodzielnie go podnosi lub puszcza ($LiftA/B$, $DropA/B$), otwiera lub zamyka drzwi ($OpenA/B$, $CloseA/B$) oraz wspólnie wynie stół na zewnetrz ($CarryOut$). Stół uznajemy za uniesiony tylko wtedy, gdy trzymają go jednocześnie obaj robotnicy. Drzwi mona obsugiwać wycznie woln rk.

Sygnatura:

$$\begin{aligned}\mathcal{F} &= \{holdA, holdB, tableUp, doorOpen, broken, outside\}, \\ \mathcal{A} &= \{LiftA, LiftB, DropA, DropB, OpenA, OpenB, CloseA, CloseB, CarryOut\}.\end{aligned}$$

Dla zwizoci poniej uywamy skrótów $a = holdA$, $b = holdB$, $u = tableUp$, $d = doorOpen$, $r = broken$, $o = outside$ i zapisujemy stan jako szóstkę (a, b, u, d, r, o) .

Dziedzina D :

Ramifikacje i fluenty nieinercyjne:

always $u \leftrightarrow (a \wedge b)$ (stół uniesiony $\Leftrightarrow$ trzymają go obaj)

noninertial u (stół zależy od robotników)

Podnoszenie i opuszczanie stou:

$LiftA$ **causes** a , $LiftB$ **causes** b (pojedynczy robotnik chwyta stół)

$DropA$ **causes** $\neg a$, $DropB$ **causes** $\neg b$ (pojedynczy robotnik puszcza stół)

Obsuga drzwi (tylko woln rk):

$OpenA$ **causes** d , $OpenB$ **causes** d (dowolny robotnik otwiera drzwi)

$CloseA$ **causes** $\neg d$, $CloseB$ **causes** $\neg d$ (dowolny robotnik zamyka drzwi)

impossible $OpenA$ if a , **impossible** $OpenB$ if b (potrzeba wolnej rki)

impossible $CloseA$ if a , **impossible** $CloseB$ if b (analogicznie dla zamykania)

Wynoszenie stou:

$CarryOut$ **causes** o if d (wyjcie tylko przez otwarte drzwi)

impossible $CarryOut$ if $\neg u$ (nie mona wynie lecego stou)

$CarryOut$ **releases** a if $\neg d$

$CarryOut$ **releases** b if $\neg d$

$CarryOut$ **releases** r if $\neg d$ (uderzenie w drzwi — chaos)

Opis stanu poczkowego i obserwacje:

initially $\neg a \wedge \neg b \wedge \neg o$ (nie trzymają, stół w rodku; d, r nieustalone)

initially $\neg d \vee \neg r$ (jeli drzwi poczkowo otwarte, to stół jeszcze nieuszkodzony)

observable $\neg r$ after $LiftA, LiftB, CarryOut$ (obserwacja — da si wynie cao)

Stany dopuszczalne Σ . Sporód $2^6 = 64$ wartociowa ograniczenie **always** $u \leftrightarrow (a \wedge b)$ wyznacza u na podstawie a, b , co daje $|\Sigma| = 2^5 = 32$ stany (swobodne a, b, d, r, o).

Zbiór Σ_0 . Dwa zdania **initially** wyznaczaj Σ_0 cznie (zgodnie z §8.1). Warunek $\neg a \wedge \neg b \wedge \neg o$ ustala trzy fluenty; u jest wyliczone ($u = F$ dla $a = b = F$). Swobodne pozostaj d, r , co daje cztery kandydatury:

$$\begin{aligned} s_1 &= (F, F, F, F, F, F), & s_2 &= (F, F, F, F, T, F), \\ s_3 &= (F, F, F, T, F, F), & s_4 &= (F, F, F, T, T, F). \end{aligned}$$

Drugie zdanie **initially** $\neg d \vee \neg r$ odrzuca wycznie s_4 (jedyne kandydat z $d = T$ i jednoczenie $r = T$). Ostatecznie

$$\Sigma_0 = \{\sigma_0, \sigma_1, \sigma_2\}, \quad \sigma_0 = s_1, \quad \sigma_1 = s_2, \quad \sigma_2 = s_3.$$

Jest to przypadek opisu czciowego (Z7): trzy stany pocztkowe, rónice si wartoci d (drzwi) i r (stan stou) poza zakresem pierwszego **initially**.

Sprawdzenie M2 dla zdania observable. Zdanie **observable** $\neg r$ **after** $LiftA, LiftB, CarryOut$ jest warunkiem poprawnoci modelu, a nie dodatkowym filtrem Σ_0 ; wymaga, by *istnia* $\sigma_0 \in \Sigma_0$ oraz lad wykonania sekwencji, którego stan kocowy spenia $\neg r$. W kadym σ_i po $LiftA, LiftB$ mamy $a = b = u = T$, wartoci d, r niezmiennione; w kroku $CarryOut$:

- gdy $d = F$ (stany σ_0, σ_1): $\neg d$ aktywuje **releases** dla a, b, r , natomiast **causes** o **if** d jest nieaktywne. Fluent r jest zwolniony, wic istnieje stan wynikowy z $r = F$.
- gdy $d = T$ (stan σ_2): **releases** nieaktywne, r pozostaje bez zmian; σ_2 ma $r = F$, wic stan kocowy te.

W kadym z trzech przypadków istnieje lad koczcy si w stanie z $\neg r$, wic M2 zachodzi — dziedzina ma model.

Poniej rozwaymy dwa procesy, które razem pokazuj wszystkie mechanizmy semantyki. Kady z nich uywamy jako $\mathbb{P}$ w osobnym komplecie kwerend.

Proces $P_1 = (\{OpenA\}, \{LiftA, DropA\}, \{LiftB\}, \{CarryOut\})$. Cel: pokaza (i) akcj singletonow z warunkow **causes**, (ii) konflikt Przypadek 1 w kroku 2, (iii) dynamiczne **impossible** aktywujce si dopiero na kocu procesu.

Krok 1: $\{OpenA\}$. Singleton — brak konfliktu. W kadym $\sigma \in \Sigma_0$ mamy $a = F$, wic **impossible** $OpenA$ **if** a jest nieaktywne.

- $\sigma_0 \rightarrow \sigma_2$ (jedyna zmiana: $d: F \rightarrow T$).
- $\sigma_1 \rightarrow \sigma_3 := (F, F, F, T, T, F)$.
- $\sigma_2 \rightarrow \sigma_2$ (d ju prawdziwe — efekt pusty, $New = \emptyset$).

Zbiór osigalny: $\{\sigma_2, \sigma_3\}$.

Krok 2: $\{LiftA, DropA\}$. W obu stanach obie akcje s indywidualnie wykonalne. $E(LiftA) = \{a\}$, $DNF = \{\{a\}\}$; $E(DropA) = \{\neg a\}$, $DNF = \{\{\neg a\}\}$. Literay a i $\neg a$ s komplementarne — **konflikt**

Przypadek 1. Dekompozycje: $\delta_1 = \{LiftA\}$, $\delta_2 = \{DropA\}$.

Wyniki (stan zapisujemy przed $\rightarrow$ po):

- z σ_2 : δ_1 daje $\sigma_4 := (T, F, F, T, F, F)$; δ_2 daje σ_2 (efekt $\neg a$ ju speniony). $Res = \{\sigma_2, \sigma_4\}$.
- z σ_3 : analogicznie $\sigma_5 := (T, F, F, T, T, F)$ oraz σ_3 . $Res = \{\sigma_3, \sigma_5\}$.

Krok 3: $\{LiftB\}$. Singleton — brak konfliktu, brak **impossible**. Akcja wymusza b ; ramifikacja **always** dopasowuje u do $a \wedge b$ (tutaj u nieinercyjny, wic nie trafia do New):

- $\sigma_2 \rightarrow \sigma_6 := (F, T, F, T, F, F)$ (u pozostaje F, bo $a = F$).
- $\sigma_3 \rightarrow \sigma_7 := (F, T, F, T, T, F)$.
- $\sigma_4 \rightarrow \sigma_8 := (T, T, T, T, F, F)$ (u wchodzi w T przez ramifikację).
- $\sigma_5 \rightarrow \sigma_9 := (T, T, T, T, T, F)$.

Krok 4: $\{CarryOut\}$. Singleton, ale **impossible** *CarryOut* if $\neg u$ blokuje akcję w stanach z $u = F$:

- σ_6, σ_7 : $u = F$ — **niewykonalne**. łańcuch Ψ na tej gazi pozostaje niezdefiniowany.
- σ_8 : $u = T, d = T$, więc **causes** o if d wymusza o , **releases** (if $\neg d$) nieaktywne. $Res = \{\sigma_{10}\}$, $\sigma_{10} := (T, T, T, T, F, T)$.
- σ_9 : analogicznie, $Res = \{\sigma_{11}\}$, $\sigma_{11} := (T, T, T, T, T, T)$.

Drzewo łańcuchów P_1 :

$$\begin{array}{l}
 \sigma_0 \rightarrow \sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_6 \rightarrow \textbf{niewykonalne} \\
 \sigma_4 \rightarrow \sigma_8 \rightarrow \sigma_{10} \text{ (SUCCESS)} \\
 \sigma_1 \rightarrow \sigma_3 \rightarrow \sigma_3 \rightarrow \sigma_7 \rightarrow \textbf{niewykonalne} \\
 \sigma_5 \rightarrow \sigma_9 \rightarrow \sigma_{11} \text{ (SUCCESS)} \\
 \sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_6 \rightarrow \textbf{niewykonalne} \\
 \sigma_4 \rightarrow \sigma_8 \rightarrow \sigma_{10} \text{ (SUCCESS)}
 \end{array}$$

Kwerendy dla P_1 :

- **necessary executable** P_1 — z każdego $\sigma \in \Sigma_0$ istnieje łańcuch kończący się „niewykonalne” (gałąź przez $\delta_2 = \{DropA\}$). **NIE**.
- **possibly executable** P_1 — z każdego $\sigma \in \Sigma_0$ istnieje łańcuch peny (gałąź przez $\delta_1 = \{LiftA\}$). **TAK**.
- **necessary o after** P_1 — na gałęziach niewykonalnych Ψ jest niezdefiniowane, więc kwerenda konieczności nie jest spełniona. **NIE**.
- **possibly o after** P_1 — stany σ_{10}, σ_{11} spełniają o . **TAK**.

Proces $P_2 = (\{LiftA, LiftB\}, \{LiftA, CarryOut\})$. Cel: pokazać (i) akcję zoon bez konfliktu w kroku 1, (ii) konflikt Przypadek 2 (**causes** vs. aktywne **releases**) w kroku 2, (iii) warunkowe **releases** rozbijając wiat na wiele stanów wynikowych.

Krok 1: $\{LiftA, LiftB\}$. $E(LiftA) = \{a\}$, $E(LiftB) = \{b\}$ — różne fluenty, adna akcja nie jest **releases**, więc **brak konfliktu** (ani Przypadek 1, ani 2). Jedna dekompozycja $\delta = \{LiftA, LiftB\}$, $E(\delta) = \{a, b\}$, $R(\delta) = \emptyset$.

Z każdego stanu $\sigma \in \Sigma_0$ otrzymujemy jeden stan wynikowy (ramifikacja wymusza $u = T$, fluent nieinercyjny):

- $\sigma_0 \rightarrow \sigma_{12} := (T, T, T, F, F, F)$.
- $\sigma_1 \rightarrow \sigma_{13} := (T, T, T, F, T, F)$.
- $\sigma_2 \rightarrow \sigma_8$ (ten sam σ_8 co w P_1).

Krok 2: $\{LiftA, CarryOut\}$. Tu dzieje się najwięcej. Analiza zależy od tego, czy d jest prawdziwe w stanie, w którym wykonujemy akcję zoon.

W stanach σ_{12}, σ_{13} ($d = F$). *CarryOut*: $u = T$, akcja wykonalna; $\neg d = T$, więc **aktywna** jest **releases** dla a, b, r , natomiast **causes** o if d jest **nieaktywne**. Zatem $E(CarryOut) = \emptyset$,

$R(\text{CarryOut}) = \{a, b, r\}$. *LiftA*: $E = \{a\}$, $\text{DNF} = \{\{a\}\}$ — fluent a występuje w DNF zarówno w efekcie **causes** *LiftA*, jak i w aktywnej **releases** *CarryOut*. **Konflikt Przypadek 2** (**causes** vs. **releases**).

Dekompozycje: $\delta_1 = \{\text{LiftA}\}$, $\delta_2 = \{\text{CarryOut}\}$.

- δ_1 w σ_{12} : a ju prawdziwe, $\text{New} = \emptyset$, $\text{Res}(\delta_1, \sigma_{12}) = \{\sigma_{12}\}$; analogicznie $\text{Res}(\delta_1, \sigma_{13}) = \{\sigma_{13}\}$.
- δ_2 w σ_{12} : $E = \emptyset$, $R = \{a, b, r\}$. Fluenty niezwolnione (d, o) musz pozosta, czyli $d = o = \text{F}$; fluenty zwolnione (a, b, r) mog si zmienia swobodnie; u wyliczane przez ramifikacj. Dla kadej z $2^3 = 8$ kombinacji (a, b, r) mamy $\text{New} = \emptyset$ (aden z fluentów inercyjnych nie wymusza zmiany). Wszystkie 8 kandydatów jest minimalnych:

$$\begin{aligned} \tau_1 &= (\text{T}, \text{T}, \text{T}, \text{F}, \text{T}, \text{F}) = \sigma_{13}, & \tau_2 &= (\text{T}, \text{T}, \text{T}, \text{F}, \text{F}, \text{F}) = \sigma_{12}, \\ \tau_3 &= (\text{T}, \text{F}, \text{F}, \text{F}, \text{T}, \text{F}), & \tau_4 &= (\text{T}, \text{F}, \text{F}, \text{F}, \text{F}, \text{F}), \\ \tau_5 &= (\text{F}, \text{T}, \text{F}, \text{F}, \text{T}, \text{F}), & \tau_6 &= (\text{F}, \text{T}, \text{F}, \text{F}, \text{F}, \text{F}), \\ \tau_7 &= (\text{F}, \text{F}, \text{F}, \text{F}, \text{T}, \text{F}) = \sigma_1, & \tau_8 &= (\text{F}, \text{F}, \text{F}, \text{F}, \text{F}, \text{F}) = \sigma_0. \end{aligned}$$

$\text{Res}(\delta_2, \sigma_{12}) = \{\tau_1, \dots, \tau_8\}$; tak samo $\text{Res}(\delta_2, \sigma_{13}) = \{\tau_1, \dots, \tau_8\}$ (zbiór zwolnionych fluentów pokrywa wszystkie fluenty, którymi oba stany si róni).

$\text{Res}(\{\text{LiftA}, \text{CarryOut}\}, \sigma_{12}) = \text{Res}(\delta_1, \sigma_{12}) \cup \text{Res}(\delta_2, \sigma_{12}) = \{\tau_1, \dots, \tau_8\}$ (8 stanów, wszystkie z $o = \text{F}$). Analogicznie dla σ_{13} .

W stanie σ_8 ($d = \text{T}$). Teraz $\neg d = \text{F}$, wic wszystkie trzy **releases** s *nieaktywne*; **causes** o if d *aktywne*. $E(\text{CarryOut}) = \{o\}$, $R(\text{CarryOut}) = \emptyset$. Z *LiftA* brak wspólnych literałów komplementarnych ani zwolnionych fluentów z DNF — **brak konfliktu**. Jedna dekompozycja $\delta = \{\text{LiftA}, \text{CarryOut}\}$, $E(\delta) = \{a, o\}$. Stan σ_8 ju spenia a ; jedyny minimalny stan wynikowy to σ_{10} (z P_1). Zatem $\text{Res}(\{\text{LiftA}, \text{CarryOut}\}, \sigma_8) = \{\sigma_{10}\}$.

Drzewo ladów P_2 :

$$\begin{aligned} \sigma_0 &\rightarrow \sigma_{12} \rightarrow \{\tau_1, \tau_2, \tau_3, \tau_4, \tau_5, \tau_6, \tau_7, \tau_8\} \quad (\text{wszystkie maj } o = \text{F}) \\ \sigma_1 &\rightarrow \sigma_{13} \rightarrow \{\tau_1, \tau_2, \tau_3, \tau_4, \tau_5, \tau_6, \tau_7, \tau_8\} \\ \sigma_2 &\rightarrow \sigma_8 \rightarrow \{\sigma_{10}\} \quad (\text{ma } o = \text{T}) \end{aligned}$$

Kady lad Ψ jest zdefiniowany — proces P_2 jest wszdzie wykonalny.

Kwerendy dla P_2 :

- **necessary executable P_2** — w adnym wle drzewa wykonania nie zachodzi blokada. **TAK**.
- **possibly executable P_2** — natychmiast z powyszego. **TAK**.
- **necessary o after P_2** — ga startujca z σ_0 koczy si w zbiorze $\{\tau_1, \dots, \tau_8\}$, w którym wszdzie $o = \text{F}$. Decyduje o tym wybór dekompozycji $\delta_2 = \{\text{CarryOut}\}$ w kroku 2 — bo $d = \text{F}$, **causes** dla o nie odpala, a **releases** nie wymusza o . **NIE**.
- **possibly o after P_2** — ga z σ_2 koczy si w σ_{10} z $o = \text{T}$ (drzwi pocztkowo otwarte, **causes** o if d *aktywne*). **TAK**.

Komplementarno procesów: P_1 prowadzi przez konflikt Przypadek 1 i aktywuje dynamiczn niewykonalno *CarryOut*, co daje negatywn odpowied na **necessary executable**; P_2 ujawnia konflikt Przypadek 2 z niedeterministyczn **releases**, która przy $d = \text{F}$ rozбивa wiat na osiem stanów wynikowych i blokuje pozytywn odpowied na **necessary o after P_2** . Zdanie **observable** peni tu rol warunku M2, wspólnego dla obu procesów, i jest zweryfikowane na ju wyznaczonym Σ_0 .

11 Podsumowanie i kierunek dalszych prac

W niniejszym opracowaniu zaprojektowano kompletny formalny aparat do opisu oraz analizy systemów dynamicznych klasy DS4. W szczególności zdefiniowano:

- **jzyk akcji**, umoliwiający formalny opis dziedziny w oparciu o akcje proste, ich efekty, warunki wykonalności oraz ograniczenia integralności,
- **semantyk operacyjn**, obejmując zarówno pojedyncze akcje, jak i akcje zoone, wraz z mechanizmem konfliktów, dekompozycji oraz minimalizacji zmian,
- **model dziedziny**, który jednoznacznie określa zbiór stanów dopuszczalnych, stanów początkowych oraz funkcję przejścia,
- **jzyk kwerend**, pozwalający formułować pytania dotyczące wykonalności procesów oraz osiągalności określonych stanów końcowych,
- **zestaw decyzji projektowych** (§8) dotyczących m.in. cznej interpretacji wielu zda **initially**, ram rozróżnienia między zdaniem wartociowym dziedziny a kwerend **necessary after**, wyboru literaowego kryterium konfliktu (w miejsce rozczności zmiennych ze specyfikacji) oraz roli Res_0 jako filtra kandydatów przed minimalizacją.

Kluczowym elementem projektu jest integracja mechanizmów znanych z języków *AR* oraz *AC* w ramach jednej spójnej semantyki. W szczególności wykorzystano: prawo inercji i ramifikacje do modelowania zmian stanu, niedeterminizm wynikający z konstrukcji **releases**, oraz mechanizm dekompozycji akcji zoonych do obsługi współbicieni. Zaproponowane podejście uogólnia prosty model rozczności zmiennych, umoliwiając bardziej elastyczne i realistyczne modelowanie konfliktów między akcjami.

Istotnym rozszerzeniem względem klasycznego modelu *AR* jest dopuszczenie wielu stanów początkowych (Σ_0), co wynika bezpośrednio z zażenia częściowej informacji (Z7). Wprowadza to dodatkowy poziom niedeterminizmu, który musi być uwzględniany przy interpretacji kwerend.

Kierunki dalszych prac.

Kolejnym etapem projektu będzie implementacja zaproponowanego modelu oraz eksperymentalna weryfikacja jego działania. Planowane prace obejmują:

- **implementację parsera języka dziedziny i kwerend** – opracowanie prostego DSL umoliwiającego wprowadzanie specyfikacji w formie tekstowej,
- **generację przestrzeni stanów** (Σ oraz Σ_0) na podstawie ograniczeń i zda początkowych,
- **implementację funkcji** Res dla akcji prostych i zoonych, w tym:
 - obliczanie Res_0 ,
 - wyznaczanie zbioru zmian New ,
 - minimalizację względem zasady inercji,
 - wykrywanie konfliktów i generację dekompozycji,
- **implementację ewaluacji procesów** (funkcja Ψ) wraz z obsługą niedeterminizmu,
- **obsługę kwerend** typu **necessary** oraz **possibly**, zarówno dla wykonalności, jak i osiągalności celów,
- **opracowanie zestawu testów i przykładów**, pozwalających zweryfikować poprawność implementacji na scenariuszach omawianych w pracy,

- opcjonalnie: **interfejs użytkownika** (np. prosty CLI lub wizualizacja drzewa stanów), ułatwiający analiz działania systemu.

Dalsze prace mog również obejmować analiz złożoności obliczeniowej zaproponowanych algorytmów oraz ewentualne optymalizacje, w szczególności w zakresie generowania dekompozycji i eksploracji przestrzeni stanów. Interesującym kierunkiem rozszerzenia byłoby także dopuszczenie bardziej złożonych form akcji (np. hierarchicznych) lub integracja z istniejącymi systemami planowania.

Podsumowując, zaproponowany model stanowi solidną podstawę zarówno do implementacji, jak i dalszych badań nad reprezentacją wiedzy w dynamicznych systemach wspólnych.